

第9章

AST自动化JS防护方案



本章介绍如何自动化地混淆JS代码,并着重讲述混淆方案的代码实现。在本章中,如果没有特别说明,均以处理以下代码为例:

```
Date.prototype.format = function(formatStr) {  
    var str = formatStr;  
    var Week = ['日', '一', '二', '三', '四', '五', '六'];  
    str = str.replace(/yyyy|YYYY/, this.getFullYear());  
    str = str.replace(/MM/, (this.getMonth() + 1) > 9 ? (this.getMonth() + 1).toString() :  
    '0' + (this.getMonth() + 1));  
    str = str.replace(/dd|DD/, this.getDate() > 9 ? this.getDate().toString() : '0' + this.  
    getDate());  
    return str;  
}  
console.log( new Date().format('yyyy-MM-dd') );  
//输出结果 2020-07-19
```

9.1 混淆前的代码处理

9.1.1 改变对象属性访问方式

对象属性访问方式有两种: `console.log` 和 `console["log"]`。在JS混淆中,通常会使用 `console["log"]` 这种对象属性访问方式。下面介绍两种访问方式在AST节点中的区别,示例如下:

```
traverse(ast, {  
    MemberExpression(path){
```



视频讲解

```
        console.log(path.node);
    }
});

/* console.log
  Node { type: 'MemberExpression', ...
    object: Node { type: 'Identifier', ... name: 'console' },
    property: Node { type: 'Identifier', ... name: 'log' },
    computed: false }
  console["log"]
  Node { type: 'MemberExpression', ...
    object: Node { type: 'Identifier', ... name: 'console' },
    property: Node { type: 'StringLiteral', ...
      extra: { rawValue: 'log', raw: '"log"' }, value: 'log' },
    computed: true }
*/
```

可以看出主要有两处差异, `console.log` 这种访问方式的 `property` 为 `Identifier`, `computed` 为 `false`; `console["log"]` 这种访问方式的 `property` 为 `StringLiteral`, `computed` 为 `true`。因此,只要遍历 `MemberExpression`(成员表达式),修改对应的属性,即可完成两种访问方式的转换,如下所示:

```
traverse(ast, {
  MemberExpression(path){
    if(t.isIdentifier(path.node.property)){
      let name = path.node.property.name;
      path.node.property = t.stringLiteral(name);
    }
    path.node.computed = true;
  },
});

/* 输出结果只截取其中一部分
  Date["prototype"]["format"] = function (formatStr) {
    ...
    str = str["replace"](/yyyy/YYYY/, this["getFullYear"]());
    ...
  };
  console["log"](new Date()["format"]('yyyy-MM-dd'));
*/
```

进行混淆之前的代码中,两种访问方式可能都有。因此修改 `property` 属性时需要判断,只处理类型为 `Identifier` 的情况。

9.1.2 JS 标准内置对象的处理

从改变访问方式后的代码中可以看到,有两个 `Date`,它们是JS中的标准内置对象,也

可以理解为系统函数,系统函数的使用必须按照规则。内置对象一般都是 window 下的属性,因此可以转化为 window["Date"] 这种形式来访问,这里的 Date 是一个字符串,可以进行加密。另外,还有一些系统自带的全局函数,它们也是 window 下的方法,可以一并处理。

JS 发展到现在,标准内置对象极其庞杂,这里只演示其中的一部分。JS 中常见的全局函数有 eval、parseInt 和 encodeURIComponent。JS 中常见的标准内置对象有 Object、Function、Boolean、Number、Math、Date、String、RegExp 和 Array 等。这些全局函数和内置对象都是 window 对象下的,在 AST 中都是标识符。因此处理代码的过程可以是遍历 Identifier 节点,把符合要求的节点替换成 object 属性为 window 的 MemberExpression,也就是需要生成 MemberExpression,它在 ts 文件中的定义为:

```
export function memberExpression(object: Expression, property: any, computed?: boolean, optional?: true | false | null): MemberExpression;
```

传入 object、property 和 computed,返回 MemberExpression。内置对象的处理代码如下:

```
traverse(ast, {
  Identifier(path) {
    let name = path.node.name;
    if ('eval' | parseInt | encodeURIComponent | Object | Function | Boolean | Number | Math | Date | String |
    RegExp | Array'.indexOf(name) !== -1) {
      path.replaceWith(t.memberExpression(t.identifier('window'), t.stringLiteral(
      (name), true));
    }
  }
});
/* 输出结果只截取其中一部分
window["Date"]["prototype"]["format"] = function (formatStr) {
  ...
  str = str["replace"](/yyyy|YYYY/, this["getFullYear"]());
  ...
};
console.log(new window["Date"]()["format"]('yyyy-MM-dd'));
*/
```

上述代码中先获取到 Identifier 的 name 属性,然后到预先准备好的字符串中搜索。如果是内置对象或全局函数,就把它替换成 object 属性为 window 的 MemberExpression。

9.2 常量与标识符的混淆

9.2.1 实现数值常量加密

代码中的数值常量可以通过遍历 NumericLiteral 节点,获取其中的 value 属性得到,然后随机生成一个数值记为 key,接着把 value 与 key 进行异或,得到加密后的数值记为



视频讲解

cipherNum,即 $\text{cipherNum} = \text{value} \wedge \text{key}$ 。此时, $\text{value} = \text{cipherNum} \wedge \text{key}$ 。因此可以生成一个 BinaryExpression 节点来等价地替换 NumericLiteral 节点。BinaryExpression 的 operator 为 ^, left 为 cipherNum, right 为 key。具体代码如下:

```
//数值常量加密
traverse(ast, {
  NumericLiteral(path){
    let value = path.node.value;
    let key = parseInt(Math.random() * (999999 - 100000) + 100000, 10);
    let cipherNum = value ^ key;
    path.replaceWith(t.binaryExpression('^', t.numericLiteral(cipherNum),
    t.numericLiteral(key)));
    //替换后的节点里也有 numericLiteral 节点,会造成死循环,因此需要加入 path.skip()
    path.skip();
  }
});

/* 输出结果只截取其中一部分
window["Date"]["prototype"]["format"] = function (formatStr) {
  ...
  str = str["replace"](/MM/, this["getMonth"]() + (345767 ^ 345766) > (423679 ^
423670) ? (this["getMonth"]() + (471098 ^ 471099))["toString"]() : '0' + (this["getMonth"]
() + (646781 ^ 646780)));
  ...
};
console["log"](new window["Date"]()["format"]('yyyy-MM-dd'));
*/
```

9.2.2 实现字符串常量加密

原始代码经过前面几节的处理,许多标识符都变成了字符串,但是这些字符串依然是明文。这一节就要对这些字符串进行加密,加密的思路较为简单。例如,代码中明文是 "prototype",加密后的值为 "cHJvdG90eXBl",解密函数名为 atob。只要生成 atob ("cHJvdG90eXBl"),然后用它替换原先的 "prototype" 即可。解密函数也需要一起放入原始代码中。在这里用 Base64 编码字符串,然后使用浏览器自带的 atob 来解密。

如果在 AST 中操作,先要遍历所有的 StringLiteral,取出其中的 value 属性进行加密,然后把 StringLiteral 节点替换为 CallExpression(调用表达式)。对于生成 CallExpression,先来看一下 CallExpression 在 ts 文件中的定义:

```
export function callExpression(callee: Expression | V8IntrinsicIdentifier, _arguments: Array
< Expression | SpreadElement | JSXNamespacedName | ArgumentPlaceholder >): CallExpression;
```

callee 表示函数名,传入一个 Identifier 即可。_arguments 表示参数,可以有多个参数,因此它是一个数组。函数最终返回 CallExpression。实现字符串常量加密的代码如下:


```
//base64Encode 就是加密字符串的函数,可以根据需求自行实现
function base64Encode(str){
    ...
}
traverse(ast, {
    StringLiteral(path){
        //生成 callExpression 参数就是字符串加密后的密文
        let encStr = t.callExpression(
            t.identifier('atob'),
            [t.stringLiteral(base64Encode(path.node.value))]);
        path.replaceWith(encStr);
        //替换后的节点里也有 StringLiteral 节点,会造成死循环,因此需要加入 path.skip()
        path.skip();
    }
});

/* 输出结果只截取其中一部分
Date[atob("cHJvdG90eXB1")][atob("Zm9ybWF0")] = function (formatStr) {
    ...
    str = str[atob("cmVwbGFjZQ==")](/yyyy|YYYY/,
    this[atob("Z2V0RnVsbFl1YXI=")]());
    ...
};
console[atob("bG9n")](new Date()
[atob("Zm9ybWF0")][atob("eXl5eS1NTS1kZA==")]);
*/
```



视频讲解

9.2.3 实现数组混淆

观察经过字符串加密以后的代码,可以看到字符串虽然加密了,但是依然在原先的位置。数组混淆要把这些字符串都提取到数组中,原先字符串的位置改为以数组下标的方式去访问数组成员。还可以提取到多个数组中,不同的数组处于不同的作用域中。在这里就只实现提取到一个数组的情况。

例如, `Date[atob("cHJvdG90eXB1")]`,把 `"cHJvdG90eXB1"` 变成数组 `arr` 的下标为 0 的成员后,原先字符串的位置就变为 `Date[atob(arr[0])]`,还需要额外生成一个数组,放入到被混淆的代码中。

AST 的处理首先是遍历 `StringLiteral` 节点,可以和字符串加密一起处理。先构造出一个结构 `atob(arr[0])`,代码如下:

```
let encStr = t.callExpression(
    t.identifier('atob'),
    [t.memberExpression(t.identifier('arr'), t.numericLiteral(index), true)];
```

这段代码构建了一个 `CallExpression`,函数名为 `atob`,参数是 `MemberExpression`。这个 `MemberExpression` 的 `object` 属性为 `arr`(就是稍后要生成的数组名),`property` 属性为数

组索引 index, 这个值当前未知。

接着把代码中的字符串放入数组中, 得到对应的索引。这里有两种方案: 不管字符串是否重复, 遍历到一个就放入数组, 然后把新的索引赋值给 index; 字符串放入数组之前, 先查询数组中是否有相同的字符串, 如果有相同的字符串, 得到原先的索引并赋值给 index。在这里采用第二种方案, 代码如下:

```
function base64Encode(str){
    ...
}
let bigArr = [];
traverse(ast, {
    StringLiteral(path){
        let cipherText = base64Encode(path.node.value);
        let bigArrIndex = bigArr.indexOf(cipherText);
        let index = bigArrIndex;
        if(bigArrIndex == -1){
            let length = bigArr.push(cipherText);
            index = length - 1;
        }
        let encStr = t.callExpression(
            t.identifier('atob'),
            [t.memberExpression(t.identifier('arr'),
                                t.numericLiteral(index), true)]);
        path.replaceWith(encStr);
    }
});
```

在 JS 中, indexOf 不止用来搜索字符串, 还可以搜索数组中某个成员是否存在, 如果存在, 则返回索引, 否则返回 -1。

在上述代码中, path.node.value 是明文字符串, 先进行加密得到 cipherText, 再去数组中搜索是否存在与 cipherText 一样的成员。如果已经存在于数组中, 那就把原先的索引赋值给 index, 然后构建函数调用表达式, 替换当前的 StringLiteral 节点。用于替换的节点中没有 StringLiteral 节点, 不会死循环, 所以不需要加入停止遍历的代码。如果不存在于数组中, 也就是 indexOf 返回的结果为 -1, 就把 cipherText 加入到数组中。数组的 push 方法可以同时加入多个成员, 它除了加入成员到数组末尾以外, 还会返回加入成员以后的数组长度, 因此 length - 1 就是数组最末尾成员的索引。

当前 bigArr 中的成员只是 JS 中的字符串, 并不是 AST 中需要的 StringLiteral 节点。因此还需要做进一步处理, 把 bigArr 中的成员都转成 stringLiteral 节点。代码如下:

```
bigArr = bigArr.map(function(v){
    return t.stringLiteral(v);
});
```

现在把 bigArr 加入到 ast 中, 也就是要先用 types 组件生成一个数组声明, 并且数组成员与 bigArr 一致, 然后加入到被混淆代码的最上面。实现代码如下:

```
bigArr = t.variableDeclarator(t.identifier('arr'), t.arrayExpression(bigArr));
bigArr = t.variableDeclaration('var', [bigArr]);
ast.program.body.unshift(bigArr);
```

program 节点相当于整个 JS 文件,它的 body 属性是一个数组,用 unshift 把 bigArr 加入到最前面。完整的数组混淆处理代码如下:

```
function base64Encode(str){
    ...
}
let bigArr = [];
traverse(ast, {
    StringLiteral(path){
        let cipherText = base64Encode(path.node.value);
        let bigArrIndex = bigArr.indexOf(cipherText);
        let index = bigArrIndex;
        if(bigArrIndex == -1){
            let length = bigArr.push(cipherText);
            index = length - 1;
        }
        let encStr = t.callExpression(
            t.identifier('atob'),
            [t.memberExpression(t.identifier('arr'),
                                t.numericLiteral(index), true)]);
        path.replaceWith(encStr);
    }
});
bigArr = bigArr.map(function(v){
    return t.stringLiteral(v);
});
bigArr = t.variableDeclarator(t.identifier('arr'), t.arrayExpression(bigArr));
bigArr = t.variableDeclaration('var', [bigArr]);
ast.program.body.unshift(bigArr);

/* 输出结果只截取其中一部分
var arr = ["chJvdG90eXB1", "Zm9ybWFO", "5Q==", "AA==", "jA==", "CQ==", "2w==",
"1A==", "bQ==", "cmVwbGFjZQ==", "Z2V0RnVsbnVsbFl1YXI=", "Z2V0TW9udGg=", "dG9TdHJpbmc=",
"MA==", "Z2V0RGF0ZQ==", "bG9n", "eXl5eS1NTS1kZA=="];
Date[atob(arr[0])][atob(arr[1])] = function (formatStr) {
    ...
    str = str[atob(arr[9])]/yyyy/YYYY/, this[atob(arr[10])()];
    ...
};
console[atob(arr[15])](new Date()[atob(arr[1])][atob(arr[16])]);
*/
```

9.2.4 实现数组乱序

经过数组混淆以后的代码,引用处的数组索引与原字符串是一一对应的。这一节要打乱这个数组的顺序。代码较为简单,传入一个数组,并且指定循环次数,每次循环都把数



视频讲解

组后面的成员放前面。代码如下：

```
(function(myArr, num){
    var xiaojianbang = function(nums){
        while(--nums){
            myArr.unshift(myArr.pop());
        }
    };
    xiaojianbang(++num);
})(bigArr, 0x10);
```

既然数组的顺序和原先不一样,那么被混淆的代码在执行之前是需要还原的。因此还需要一段还原数组顺序的代码。这里的代码逆向编写,即循环同样的次数,把数组前面的成员放后面。代码如下:

```
(function(myArr, num) {
    var?xiaojianbang = function (nums) {
        while (-- nums) {
            myArr.push(myArr.shift());
        }
    };
    xiaojianbang(++num);
})(arr, 0x10);
```

还原顺序的代码要和被混淆的代码放在一起,把还原数组顺序的代码保存到新文件 demoFront.js 中,读取该文件并解析为 astFront。由于还原数组顺序的代码最外层只有一个函数节点,因此取出其中的 astFront.program.body[0]放入到被混淆代码中的 body 中。代码如下:

```
//读取还原数组顺序的函数,并解析为 astFront
const jscodeFront = fs.readFileSync("./demoFront.js", {
    encoding: "utf-8"
});
let astFront = parser.parse(jscodeFront);
//把还原数组顺序的代码加入到被混淆代码的 ast 中
ast.program.body.unshift(astFront.program.body[0]);
//构建数组声明语句
bigArr = t.variableDeclarator(t.identifier('arr'), t.arrayExpression(bigArr));
bigArr = t.variableDeclaration('var', [bigArr]);
//把数组放到被混淆代码的 ast 最前面
ast.program.body.unshift(bigArr);
```

从上述代码中可以看出,数组混淆中生成的数组,需要放到还原数组顺序的代码前面。

9.2.5 实现十六进制字符串

实现数组顺序还原的代码是没有经过混淆的。代码中标识符的混淆,可以最后一起处

理。其中,像 push、shift 这些方法可以转为字符串,由于这些代码处于还原数组顺序的代码中,因此无法把它们提取到大数组中。在这里就简单地把它们编码成十六进制字符串,如下所示:

```
function hexEnc(code) {
  for (var hexStr = [], i = 0, s; i < code.length; i++) {
    s = code.charCodeAt(i).toString(16);
    hexStr += "\\x" + s;
  }
  return hexStr
}

traverse(astFront, ?{
  MemberExpression(path){
    if(t.isIdentifier(path.node.property)){
      let name = path.node.property.name;
      path.node.property = t.stringLiteral(hexEnc(name));
    }
    path.node.computed = true;
  }
});
let code = generator(astFront).code;
console.log(code);

/* 处理后的输出结果
(function (myArr, num) {
  var xiaojianbang = function (nums) {
    while ( -- nums) {
      myArr["\\x70\\x75\\x73\\x68"](myArr["\\x73\\x68\\x69\\x66\\x74"]());
    }
  };
  xiaojianbang(++num);
})(arr, 0x12);
*/
```

上述代码遍历 MemberExpression 节点,将 property 的值进行 hex 编码,然后 usingLiteral 节点替换 property,并把 computed 改为 true,最后输出的结果有两个转义字符,其实是 Babel 在处理节点时,自动转义了反斜杠。在转成代码后,把它们替换掉即可,即 code=code.replace(/\\\\x/g, '\\x')。

另外,unicode 字符串的实现方式与本节一致,把 hex 编码的函数换成 unicode 编码的函数即可。

9.2.6 实现标识符混淆

一般情况下,标识符都是有语义的,根据标识符名可以大致推测出代码意图,因此标识符混淆很有必要。在 8.4.5 节介绍过一个简单的标识符混淆方案,这个方案的缺点是整个文件中不同标识符的名称都是不一样的。实际开发中,可以让各个函数之间的局部标识符



视频讲解

名相同,函数内的局部标识符名还可以与没有引用到的全局标识符名相同,这样做更具迷惑性,本节主要实现这种方案。

实现这种方案需要用到一个方法,即 `scope.getOwnBinding`。该方法可以用来获取属于当前节点自己的绑定。例如,在 `Program` 节点下,使用 `getOwnBinding` 就可以获取到全局标识符名,而函数内局部标识符名不会被获取到。要获取到局部标识符名可以遍历函数节点,在 `FunctionExpression` 与 `FunctionDeclaration` 节点下,使用 `getOwnBinding` 会获取到函数自身定义的局部标识符名,而不会获取到全局标识符名。遍历这三种节点,执行同一个重命名方法,代码如下:

```
traverse(ast, {
  'Program|FunctionExpression|FunctionDeclaration'(path) {
    renameOwnBinding(path);
  }
});
```

`renameOwnBinding` 如何实现呢? 来看下面这段代码:

```
function renameOwnBinding(path) {
  let OwnBindingObj = {}, globalBindingObj = {}, i = 0;
  path.traverse({
    Identifier(p) {
      let name = p.node.name;
      let binding = p.scope.getOwnBinding(name);
      binding ? (OwnBindingObj[name] = binding) : (globalBindingObj[name] = 1);
    }
  });
}
```

上述代码先遍历当前节点中所有的 `Identifier`, 得到 `Identifier` 的 `name` 属性, 通过 `getOwnBinding` 判断是否为当前节点自己的绑定。如果 `binding` 为 `undefined`, 则表示是其父级函数标识符或者全局标识符, 就将该标识符名作为属性名, 放入到 `globalBindingObj` 对象中; 如果 `binding` 存在, 则表示是当前节点自己的绑定, 就将该标识符作为属性名, `binding` 作为属性值, 放入到 `OwnBindingObj` 对象中。

这里有以下四点需要注意。

① `globalBindingObj` 中存放的不是所有全局标识符, 而是当前节点引用到的全局标识符。因为重命名标识符时, 不能与引用到的全局标识符重名, 需要进行判断。至于没有引用到的全局标识符名, 重名才更具迷惑性, 最后使用的还是当前节点定义的局部标识符。

② `OwnBindingObj` 中存储对应标识符的 `binding`。因为重命名标识符时, 需要使用 `binding.scope.rename` 方法。

③ 把标识符名作为对象的属性名。如果一个 `Identifier` 有多处引用, 就会遍历到多个, 但实际上只需要调用一次 `scope.rename` 即可完成所有引用处的重命名。而对象属性名具有唯一性, 可以只保留最后一个同名标识符。

④ 把 `ast` 先转成代码, 再次进行解析后, 再进行标识符混淆。前面章节介绍过修改

AST 节点时,如果使用 Path 对象的方法,Babel 会更新 Path 信息。但是实际应用中并不能做到全部使用 Path 对象的方法。例如,用 types 组件生成新节点,由于 types 组件生成的是节点,并不是 Path 对象,那么 binding 是没有来源的。

接下来遍历 OwnBindingObj 对象中的属性,进行重命名,代码如下:

```
for(let oldName in OwnBindingObj) {
  do {
    var newName = '_0x2ba6ea' + i++;
  } while(globalBindingObj[newName]);
  OwnBindingObj[oldName].scope.rename(oldName, newName);
}
```

上述代码中使用 do...while 循环来随机取一个标识符名,直到与当前节点引用到的全局标识符名不一样时,进行重命名。这段代码看起来没有问题,实际上 getOwnBinding 会取到当前函数中的子函数的局部标识符。不影响最后结果,因为在当前函数中对子函数的局部标识符进行重命名,当遍历到子函数时,还会再次重命名,在这里使用比较标识符作用域的方法来避免这种情况。完整的混淆标识符的代码如下:

```
function renameOwnBinding(path) {
  let OwnBindingObj = {}, globalBindingObj = {}, i = 0;
  path.traverse({
    Identifier(p)? {
      let name = p.node.name;
      let binding = p.scope.getOwnBinding(name);
      binding && generator(binding.scope.block).code == path + '?
      (OwnBindingObj[name] = binding) : (globalBindingObj[name] = 1);
    }
  });
  for(let oldName in OwnBindingObj) {
    do {
      var newName = '_0x2ba6ea' + i++;
    } while(globalBindingObj[newName]);
    OwnBindingObj[oldName].scope.rename(oldName, newName);
  }
  traverse(ast, {
    'Program|FunctionExpression|FunctionDeclaration'(path) {
      renameOwnBinding(path);
    }
  });
};

/* 以处理 8.4 节中的代码为例,输出以下结果
const _0x2ba6ea0 = 1000;
let _0x2ba6ea1 = 2000;
let _0x2ba6ea2 = {
  name: 'xiaojianbang',
  add: function (_0x2ba6ea0) {
```

```

        _0x2ba6ea0 = 400;
        _0x2ba6ea1 = 300;
        let _0x2ba6ea3 = 700;
        function _0x2ba6ea4(_0x2ba6ea0) {
            let _0x2ba6ea1 = 600;
        }
        _0x2ba6ea4();
        return _0x2ba6ea0 + _0x2ba6ea0 + _0x2ba6ea1 + 1000 + _0x2ba6ea2.name;
    }
};
_0x2ba6ea2.add(100);
//混淆后的代码,控制台运行结果 "2100xiaojianbang" 与原先一致
*/

```

在 binding & . generator(binding, scope, block). code == path + '' 这段代码中, binding, scope, block 表示 binding 的作用域, path + '' 表示把当前节点转为代码。这里的 path 指的是 Program、FunctionExpression 和 FunctionDeclaration 中任意一个。子函数的参数作用域转为代码后是子函数本身,与当前节点转为代码后不一致。

9.2.7 标识符的随机生成



视频讲解

在 9.2.6 节中,重命名标识符时使用固定的 _0x2ba6ea 加上一个自增的数字来作为新的标识符名。这一节将使用大写字母 O、小写字母 o 和数字 0 这三个字符来组成标识符名。把 9.2.6 节中 var newName = '_0x2ba6ea' + i ++; 这句代码改成 var newName = generatorIdentifier(i ++); 以下是 generatorIdentifier 的实现代码:

```

function generatorIdentifier(decNum){
    let flag = ['O', 'o', '0'];
    let retval = [];
    while(decNum > 0){
        retval.push(decNum % 3);
        decNum = parseInt(decNum / 3);
    }
    let Identifier = retval.reverse().map(function(v){
        return flag[v]
    }).join('');
    Identifier.length < 6 ? (Identifier = ('000000' + Identifier).substr(-6)) :
    Identifier[0] == '0' && (Identifier = 'O' + Identifier);
    return Identifier;
}

/* 以处理 8.4 节中的代码为例,输出以下结果
const 000000 = 1000;
let 00000o = 2000;
let 000000 = {
    name: 'xiaojianbang',

```



```

add: function (000000) {
    000000 = 400;
    00000o = 300;
    let 0000o0 = 700;
    function 0000oo(000000) {
        let 00000o = 600;
    }
    0000oo();
    return 000000 + 000000 + 00000o + 1000 + 000000.name;
}

000000.add(100);
//混淆后的代码,控制台运行结果 "2100xiaojianbang" 与原先一致
*/

```

它本质上就是将十进制转为三进制,然后把 0、1、2 分别用大写字母 O、小写字母 o 和数字 0 来替换。retval 数组中存的是余数,经过数组的 reverse 方法倒序后,就得到十进制转为三进制的结果,只是这时里面的数字是 012。接着通过数组的 map 遍历数组成员 012,分别替换成大写字母 O、小写字母 o 和数字 0。最后用数组的 join 把数组拼接成字符串。为了使生成的标识符整齐,对于长度小于 6 的标识符,可以用大写字母 O 或者小写字母 o 补全位数;对于长度大于或等于 6 并且第一个字符是数字 0 的标识符,则往前面补一个大写字母 O 或者小写字母 o(标识符不能以数字开头)。



视频讲解

9.3 代码块的混淆

9.3.1 二项式转函数花指令

花指令用来尽可能地隐藏原代码的真实意图,并且有多种实现方案。本节要介绍如何将二项式转为函数。例如,把 $c+d$ 转换为如下形式:

```

function xxx(a, b){
    return a + b;
}
xxx(c, d);

```

不止二项式,代码中的函数调用表达式也可以处理成类似的花指令,例如 $c(d)$ 可以转换成如下形式:

```

function xxx(a, b){
    return a(b);
}
xxx(c, d);

```

在这里只实现二项式的情况。从 AST 的角度去考虑如何实现这个方案,分为以下四步:

- ① 遍历 BinaryExpression 节点,取出 operator、left 和 right。
- ② 生成一个函数,函数名不能与当前节点中的标识符冲突,参数可以固定为 a 和 b,返回语句中的运算符与 operator 一致。
- ③ 找到最近的 BlockStatement 节点,将生成的函数加入到 body 数组中的最前面。
- ④ 把原先的 BinaryExpression 节点替换为 CallExpression, callee 是函数名, _arguments 是二项式的 left 和 right。

上述实现方案中,标识符名可以随机设置,因为最后进行标识符混淆时,会处理成相似的名字。花指令的目的是增加代码量,隐藏代码的真实意图。因此每遍历到一个 operator,都可以生成不同名字的函数,不需要去判断是哪种 operator,因为并不是一种 operator 生成一个函数。实现的代码如下:

```
traverse(ast, {
  BinaryExpression(path){
    let operator = path.node.operator;
    let left = path.node.left;
    let right = path.node.right;
    let a = t.identifier('a');
    let b = t.identifier('b');
    let funcNameIdentifier = path.scope.generateUidIdentifier('xxx');
    let func = t.functionDeclaration(
      funcNameIdentifier,
      [a, b],
      t.blockStatement([t.returnStatement(
        t.binaryExpression(operator, a, b)
      )]));
    let BlockStatement = path.findParent(
      function(p){return p.isBlockStatement();});
    BlockStatement.node.body.unshift(func);
    path.replaceWith(t.callExpression(funcNameIdentifier, [left, right]));
  }
});
```

/* 为了更清晰,不加入其他混淆

```
Date.prototype.format = function (formatStr) {
  function _xxx5(a, b) {
    return a + b;
  }
  function _xxx4(a, b) {
    return a + b;
  }
  function _xxx3(a, b) {
    return a + b;
  }
  function _xxx2(a, b) {
    return a + b;
  }
  function _xxx(a, b) {
```

```

        return a > b;
    }
    ...
    str = str.replace(/MM/, _xxx(_xxx2(this.getMonth(), 1), 9) ? _xxx3(this.getMonth(),
1).toString() : _xxx4('0', _xxx5(this.getMonth(), 1)));
    ...
    return str;
};
console.log(new Date().format('yyyy-MM-dd'));
* /

```



视频讲解

9.3.2 代码的逐行加密

这种方案的原理跟字符串加密是一样的,不过需要先把代码转化为字符串,再把字符串加密后的密文传入解密函数、解密出明文,然后通过 eval 执行代码。实现代码如下:

```

traverse(ast, {
    FunctionExpression(path){
        let blockStatement = path.node.body;
        let Statements = blockStatement.body.map(function(v){
            if(t.isReturnStatement(v)) return v;
            let code = generator(v).code;
            let cipherText = base64Encode(code);
            let decryptFunc = t.callExpression(t.identifier('atob'), [t.stringLiteral
(cipherText)]);
            return t.expressionStatement(t.callExpression(t.identifier('eval'),
[decryptFunc]));
        });
        path.get('body').replaceWith(t.blockStatement(Statements));
    }
});

/*
Date.prototype.format = function (formatStr) {
    eval(atob("dmFyIHNoCiA9IGZvcmlhdFN0cjs="));
    eval(atob("dmFyIFdlZWsgPSBhbjUUnLCAnACcsICeMJywgJwknLCAn2ycsICeUJywgJ20nXTs="));
    eval(atob("c3RyID0gc3RyLnJlcGxhY2UoL3l5eXl8WVlZW58sIHRoaXMuz2V0RnVsbFllYXIoKSk7"));
    ...
    return str;
};
console.log(new Date().format('yyyy-MM-dd'));
* /

```

这段代码处理步骤如下:

(1) 遍历 FunctionExpression 节点。其中, path, node, body 即 BlockStatement 节点。blockStatement.body 是一个数组,里面是函数的代码行,它的每一个成员分别对应函数的

每一行语句,然后使用数组的 `map` 方法对每一行语句分别处理。

(2) 如果是返回语句,就不做处理,直接返回原语句。单行加密时不能加密返回语句;整个函数加密时,函数中可以有返回语句。

(3) 使用 `let code = generator(v).code`;将语句转变成字符串。

(4) 使用 `let cipherText = base64Encode(code)`;对字符串进行加密。

(5) 构建 `atob('xxxx')` 形式的代码, `atob` 是解密函数名。解密函数如果不是系统自带的,还需要把解密函数一起放入代码中。需要生成 `CallExpression`, `callee` 为解密函数名,参数为加密后的字符串常量,赋值给 `decryptFunc`。

(6) 构建 `eval(atob('xxxx'))` 形式的代码,需要生成 `CallExpression`, `callee` 为 `eval`,参数为上一步生成的 `decryptFunc`,最后用 `expressionStatement` 包裹。

(7) 当函数中所有语句处理完后,构建新的 `BlockStatement` 替换原有的即可。

代码逐行加密的方案不建议大规模应用,会导致标志太明显。可以只隐藏其中几句代码或某些变量的关键赋值位置。自动化处理过程中无法知道一段代码是否为关键代码,除非用户在代码中加入标记,表示要隐藏哪几句代码。这个标记可以是注释,例如,可以在原始代码中某一句代码后面加入注释 `Base64Encrypt`。解析以下节点,查看发生的变化:

```
str = str.replace(/yyyy|YYYY/, this.getFullYear()); //Base64Encrypt

Node {
  type: 'ExpressionStatement',
  ...
  expression: Node {
    type: 'AssignmentExpression',
    ...
    operator: '=',
    left: Node { type: 'Identifier', ... name: 'str' },
    right: Node { type: 'CallExpression', ... }
  },
  trailingComments: [{
    type: 'CommentLine',
    value: 'Base64Encrypt',
    ...
  }]
}
```

从上述结构中可以看出,生成了一个 `trailingComments` 节点,它是行尾注释。因此只要在逐行解析代码时,添加判断语句,如果有注释且为 `Base64Encrypt`,就进行加密,如下所示:

```
...
if(t.isReturnStatement(v)) return v;
if(!(v.trailingComments && v.trailingComments[0].value == 'Base64Encrypt')) return v;
delete v.trailingComments;
```

```

let code = generator(v).code;
...

/*
  Date.prototype.format = function (formatStr) {
    var str = formatStr;
    var Week = ['日', '一', '二', '三', '四', '五', '六'];
    eval(atob("c3RyID0gc3RyLnJlcGxhY2UoL3l5eXl8WVlZW58sIHROaXMuzZV0RnVsbFl1YXIoKSk7"));
    //Base64Encrypt
    ...
    return str;
  };
  console.log(new Date().format('yyyy-MM-dd'));
*/

```

delete 是 JS 自带的,用来删除对象属性,加密之前把注释删掉。使用 JS 自带的 delete 而不是 path.remove,是因为 v 和 v.trailingComments 都是 Node 对象,不是 Path 对象。



视频讲解

9.3.3 代码的逐行 ASCII 码混淆

这种方案的原理与代码的逐行加密没有太大差别,只不过去掉了字符串加密函数,改用 charCodeAt 将字符串串转到 ASCII 码,把解密函数换成 String.fromCharCode,最后使用 eval 函数来执行字符串代码。具体实现代码如下:

```

traverse(ast, {
  FunctionExpression(path){
    let blockStatement = path.node.body;
    let Statements = blockStatement.body.map(function(v){
      if(t.isReturnStatement(v)) return v;
      if(!{v.trailingComments && v.trailingComments[0].value == 'ASCIIEncrypt'})
    return v;

    delete v.trailingComments;
    let code = generator(v).code;
    let codeAscii = [].map.call(code, function(v){
      return t.numericLiteral(v.charCodeAt(0));
    });
    let decryptFuncName = t.memberExpression(t.identifier('String'), t.identifier('fromCharCode'));
    let decryptFunc = t.callExpression(decryptFuncName, codeAscii);
    return t.expressionStatement(t.callExpression(t.identifier('eval'),
    [decryptFunc]));
  });
  path.get('body').replaceWith(t.blockStatement(Statements));
});
}
});

/*

```

```
Date.prototype.format = function (formatStr) {
    eval(String.fromCharCode(118, 97, 114, 114, 32, 115, 116, 114, 32, 61, 32, 102, 111, 114,
109, 97, 116, 83, 116, 114, 59));
    //ASCIIEncrypt
    var Week = ['日', '一', '二', '三', '四', '五', '六'];
    ...
    return str;
};
console.log(new Date().format('yyyy-MM-dd'));
* /
```

字符串在JS中也是数组,不过是只读的。但是它不能直接调用数组的 map,所以上述代码中用[].map.call来间接调用,作用是一样的。把字符串中的每个成员转成ASCII码之后,用NumericLiteral节点来包裹,然后生成一个MemberExpression节点,String.fromCharCode作为解密函数名。

不管是代码逐行加密还是代码逐行ASCII混淆,都要在标识符混淆之后。由此可见,在做标识符混淆时,应该处理原始代码中的eval和Function。

9.4 完整的代码与处理后的效果



视频讲解

本章前三节介绍了多种混淆方案的实现方式。其中有些方案联合使用时,需要修改部分代码,本节介绍完整的代码以及完整处理后的效果。

```
const parser = require("@babel/parser");
const traverse = require("@babel/traverse").default;
const t = require("@babel/types");
const generator = require("@babel/generator").default;
const fs = require('fs');

//把混淆方案的相关实现方法封装成类
function ConfoundUtils(ast, encryptFunc){
    this.ast = ast;
    this.bigArr = [];
    //接收传进来的函数,用于字符串加密
    this.encryptFunc = encryptFunc;
}

//改变对象属性访问方式,console.log 改为 console["log"]
ConfoundUtils.prototype.changeAccessMode = function (){
    traverse(this.ast, {
        MemberExpression(path){
            if(t.isIdentifier(path.node.property)){
                let name = path.node.property.name;
                path.node.property = t.stringLiteral(name);
            }
            path.node.computed = true;
        },
    },
```

```

    });
}
//标准内置对象的处理
ConfoundUtils.prototype.changeBuiltinObjects = function () {
    traverse(this.ast, {
        Identifier(path) {
            let name = path.node.name;
            if ('eval' | parseInt | encodeURIComponent | Object | Function | Boolean | Number | Math | Date | String |
RegExp | Array'.indexOf(name) != -1) {
                path.replaceWith(t.memberExpression(t.identifier('window'),
t.stringLiteral(name), true));
            }
        }
    });
}
//数值常量加密
ConfoundUtils.prototype.numericEncrypt = function () {
    traverse(this.ast, {
        NumericLiteral(path) {
            let value = path.node.value;
            let key = parseInt(Math.random() * (999999 - 100000) + 100000, 10);
            let cipherNum = value ^ key;
            path.replaceWith(t.binaryExpression('^',
t.numericLiteral(cipherNum),
t.numericLiteral(key)));
            path.skip();
        }
    });
}
//字符串加密与数组混淆
ConfoundUtils.prototype.arrayConfound = function () {
    let bigArr = [];
    let encryptFunc = this.encryptFunc;
    traverse(this.ast, {
        StringLiteral(path) {
            let bigArrIndex = bigArr.indexOf(encryptFunc(path.node.value));
            let index = bigArrIndex;
            if (bigArrIndex == -1) {
                let length = bigArr.push(encryptFunc(path.node.value));
                index = length - 1;
            }
            let encStr = t.callExpression(
t.identifier('atob'),
[t.memberExpression(t.identifier('arr'),
t.numericLiteral(index), true)]);
            path.replaceWith(encStr);
        }
    });
    bigArr = bigArr.map(function (v) {

```

```
        return t.stringLiteral(v);
    });
    this.bigArr = bigArr;
}
//数组乱序
ConfoundUtils.prototype.arrayShuffle = function () {
    (function (myArr, num) {
        var xiaojianbang = function (nums) {
            while (--nums) {
                myArr.unshift(myArr.pop());
            }
        };
        xiaojianbang(++num);
    })(this.bigArr, 0x10);
}
//二项式转函数花指令
ConfoundUtils.prototype.binaryToFunc = function () {
    traverse(this.ast, {
        BinaryExpression(path) {
            let operator = path.node.operator;
            let left = path.node.left;
            let right = path.node.right;
            let a = t.identifier('a');
            let b = t.identifier('b');
            let funcNameIdentifier = path.scope.generateUidIdentifier('xxx');
            let func = t.functionDeclaration(
                funcNameIdentifier,
                [a, b],
                t.blockStatement([t.returnStatement(
                    t.binaryExpression(operator, a, b)
                )]));
            let BlockStatement = path.findParent(
                function (p) { return p.isBlockStatement(); });
            BlockStatement.node.body.unshift(func);
            path.replaceWith(t.callExpression(
                funcNameIdentifier, [left, right]));
        }
    });
}
//十六进制字符串
ConfoundUtils.prototype.stringToHex = function () {
    function hexEnc(code) {
        for (var hexStr = [], i = 0, s; i < code.length; i++) {
            s = code.charCodeAt(i).toString(16);
            hexStr += "\\x" + s;
        }
        return hexStr
    }
    traverse(this.ast, {
```



```

        MemberExpression(path){
            if(t.isIdentifier(path.node.property)){
                let?name? = ?path.node.property.name;
                path.node.property? = ?t.stringLiteral(hexEnc(name));
            }
            path.node.computed? = ?true;
        }
    });
}
//标识符混淆
ConfoundUtils.prototype.renameIdentifier = function () {
    //标识符混淆之前先转成代码再解析,确保新生成的一些节点被解析到
    let code = generator(this.ast).code;
    let newAst = parser.parse(code);
    //生成标识符
    function generatorIdentifier(decNum){
        let arr = ['0', 'o', '0'];
        let retval = [];
        while(decNum > 0){
            retval.push(decNum % 3);
            decNum = parseInt(decNum / 3);
        }
        let Identifier = retval.reverse().map(function(v){
            return arr[v]
        }).join('');
        Identifier.length < 6 ? (Identifier = ('000000' + Identifier).substr(-6)) :
        Identifier[0] == '0' && (Identifier = '0' + Identifier);
        return Identifier;
    }
    function?renameOwnBinding(path){
        let OwnBindingObj = {}, globalBindingObj = {}, i = 0;
        path.traverse({
            Identifier(p){
                let name = p.node.name;
                let binding = p.scope.getOwnBinding(name);
                binding && generator(binding.scope.block).code == path + '' ?
                (OwnBindingObj[name] = binding) : (globalBindingObj[name] = 1);
            }
        });
        for(let oldName in OwnBindingObj){
            do{
                var newName = generatorIdentifier(i++);
            }while(globalBindingObj[newName]);
            OwnBindingObj[oldName].scope.rename(oldName, newName);
        }
    }
    traverse(newAst, {
        'Program|FunctionExpression|FunctionDeclaration'(path) {
            renameOwnBinding(path);
        }
    });
}

```

```

    }
  });
  this.ast = newAst;
}
//指定代码行加密
ConfoundUtils.prototype.appointedCodeLineEncrypt = function () {
  traverse(this.ast, {
    FunctionExpression(path) {
      let blockStatement = path.node.body;
      let Statements = blockStatement.body.map(function (v) {
        if (t.isReturnStatement(v)) return v;
        if (!(v.trailingComments && v.trailingComments[0].value == 'Base64Encrypt'))
      ));
      delete v.trailingComments;
      let code = generator(v).code;
      let cipherText = base64Encode(code);
      let decryptFunc = t.callExpression(t.identifier('atob'),
        [t.stringLiteral(cipherText)]);
      return t.expressionStatement(
        t.callExpression(t.identifier('eval'), [decryptFunc]));
    });
    path.get('body').replaceWith(t.blockStatement(Statements));
  });
}
//指定代码行 ASCII 码混淆
ConfoundUtils.prototype.appointedCodeLineAscii = function () {
  traverse(this.ast, {
    FunctionExpression(path) {
      let blockStatement = path.node.body;
      let Statements = blockStatement.body.map(function (v) {
        if (t.isReturnStatement(v)) return v;
        if (!(v.trailingComments && v.trailingComments[0].value == 'ASCIIEncrypt'))
      ));
      delete v.trailingComments;
      let code = generator(v).code;
      let codeAscii = [].map.call(code, function (v) {
        return t.numericLiteral(v.charCodeAt(0));
      });
      let decryptFuncName = t.memberExpression(
        t.identifier('String'), t.identifier('fromCharCode'));
      let decryptFunc = t.callExpression(decryptFuncName, codeAscii);
      return t.expressionStatement(
        t.callExpression(t.identifier('eval'), [decryptFunc]));
    });
    path.get('body').replaceWith(t.blockStatement(Statements));
  });
}
}

```

```

//构建数组声明语句,加入到 ast 最前面
ConfoundUtils.prototype.unshiftArrayDeclaration = function(){
    this.bigArr = t.variableDeclarator(t.identifier('arr'), t.arrayExpression(this.
bigArr));
    this.bigArr = t.variableDeclaration('var', [this.bigArr]);
    this.ast.program.body.unshift(this.bigArr);
}
//拼接两个 ast 的 body 部分
ConfoundUtils.prototype.astConcatUnshift = function(ast){
    this.ast.program.body.unshift(ast);
}
ConfoundUtils.prototype.getAst = function(){
    return this.ast;
}
//Base64 编码
function base64Encode(e) {
    var r, a, c, h, o, t, base64EncodeChars =
'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/';
    for (c = e.length, a = 0, r = ''; a < c;) {
        if (h = 255 & e.charCodeAt(a++), a == c) {
            r += base64EncodeChars.charAt(h >> 2),
            r += base64EncodeChars.charAt((3 & h) << 4),
            r += '=';
            break
        }
        if (o = e.charCodeAt(a++), a == c) {
            r += base64EncodeChars.charAt(h >> 2),
            r += base64EncodeChars.charAt((3 & h) << 4 | (240 & o) >> 4),
            r += base64EncodeChars.charAt((15 & o) << 2),
            r += '=';
            break
        }
        t = e.charCodeAt(a++),
        r += base64EncodeChars.charAt(h >> 2),
        r += base64EncodeChars.charAt((3 & h) << 4 | (240 & o) >> 4),
        r += base64EncodeChars.charAt((15 & o) << 2 | (192 & t) >> 6),
        r += base64EncodeChars.charAt(63 & t)
    }
    return r
}
}
function main(){
    //读取要混淆的代码
    const jscode = fs.readFileSync("./demo.js", {
        encoding: "utf-8"
    });
    //读取还原数组乱序的代码
    const jscodeFront = fs.readFileSync("./demoFront.js", {
        encoding: "utf-8"
    });
}

```

```

//把要混淆的代码解析成 ast
let ast = parser.parse(jscode);
//把还原数组乱序的代码解析成 astFront
let astFront = parser.parse(jscodeFront);
//初始类,传递自定义的加密函数进去
let confoundAst = new ConfoundUtils(ast, base64Encode);
let confoundAstFront = new ConfoundUtils(astFront);
//改变对象属性访问方式
confoundAst.changeAccessMode();
//标准内置对象的处理
confoundAst.changeBuiltinObjects();
//二项式转函数花指令
confoundAst.binaryToFunc()
//字符串加密与数组混淆
confoundAst.arrayConfound();
//数组乱序
confoundAst.arrayShuffle();

//还原数组顺序代码,改变对象属性访问方式,对其中的字符串进行十六进制编码
confoundAstFront.stringToHex();
astFront = confoundAstFront.getAst();
//先把还原数组顺序的代码,加入到被混淆代码的 ast 中
confoundAst.astConcatUnshift(astFront.program.body[0]);

//再生成数组声明语句,并加入到被混淆代码的最开始处
confoundAst.unshiftArrayDeclaration();
//标识符重命名
confoundAst.renameIdentifier();
//指定代码行的混淆,需要放到标识符混淆之后
confoundAst.appointedCodeLineEncrypt();
confoundAst.appointedCodeLineAscii();
//数值常量混淆
confoundAst.numericEncrypt();
ast = confoundAst.getAst();
//ast 转为代码
code = generator(ast).code;
//混淆的代码中,如果有十六进制字符串加密,ast 转成代码以后会有多余的转义字符,需要替
换掉
code = code.replace(/\\\\x/g, '?\\x');
console.log(code);
}
main();

```

对上述代码进行以下三点说明。

- (1) demo.js 中存放的代码用来测试混淆方案,代码为原始代码。
- (2) demoFront.js 中存放的代码用于还原数组顺序,代码在 9.2.4 节中。
- (3) 如果不使用浏览器自带的 atob 作为解密函数,就需要实现对应的 Base64 解码函数,然后与还原数组顺序的代码一起加入。

用上述代码处理原始代码,输出结果为:

```
var 000000 = [{"Zm9ybWF0", "5Q==", "AA==", "jA==", "CQ==", "2w==", ... ]};
(function (000000, 000000) {
    var 000000 = function (000000) {
        while (--000000) {
            000000["\x70\x75\x73\x68"] (000000["\x73\x68\x69\x66\x74"] ());
        }
    };
    000000(++000000);
}) (000000, 450413 ^ 450429);
window[atob(000000[409086 ^ 409086])] [atob(000000[221471 ^ 221470])] [atob(000000[844722 ^ 844720])] = function (000000) {
    function 000000(000000, 000000) {
        return 000000 + 000000;
    }
    function 000000(000000, 000000) {
        return 000000 > 000000;
    }
    function 000000(000000, 000000) {
        return 000000 + 000000;
    }
    function 000000(000000, 000000) {
        return 000000 + 000000;
    }
    function 000000(000000, 000000) {
        return 000000 + 000000;
    }
    function 000000(000000, 000000) {
        return 000000 + 000000;
    }
    function 000000(000000, 000000) {
        return 000000 + 000000;
    }
    function 000000(000000, 000000) {
        return 000000 > 000000;
    }
}
eval(String.fromCharCode(425871 ^ 425977, 118436 ^ 118469, ... ));
var 000000 = [atob(000000[346952 ^ 346955]), atob(000000[468233 ^ 468237]), ...];
eval(atob("T09PT08wID0gT09PT08wW2F0b2IoT09PT09PWzEwXSlldKC95eXl ... "));

000000 = 000000[atob(000000[307479 ^ 307485])] (/MM/, 000000(000000(this[atob(000000[416357 ^ 416361]] ()), 139472 ^ 139473), 765252 ^ 765261) ? 000000(this[atob(000000[427063 ^ 427067]] ()), 696913 ^ 696912)[atob(000000[228455 ^ 228458])] ()): 000000(atob(000000[736284 ^ 736274]), 000000(this[atob(000000[367113 ^ 367109]] ()), 387104 ^ 387105)));

000000 = 000000[atob(000000[543343 ^ 543333])] (/dd/ DD/, 000000(this[atob(000000[123603 ^ 123612]] ()), 826697 ^ 826688) ? this[atob(000000[397862 ^ 397865])] ())[atob(000000[354736 ^ 354749]] ()): 000000(atob(000000[837968 ^ 837982]), this[atob(000000[177606 ^ 177609]] ()));

return 000000;
```

```
};  
console[atob(000000[275638 ^ 275622])](new window[atob(000000[465231 ^ 465231]))([atob  
(000000[325132 ^ 325134])](atob(000000[563198 ^ 563183])));
```

9.5 代码执行逻辑的混淆

9.5.1 实现流程平坦化

本节采用 6.3.1 节的方案来实现流程平坦化,把以下代码放到 AST Explorer 网站中解析,对照网站来实现相应的流程平坦化,如下所示:

```
Date.prototype.format = function (formatStr) {  
    let _array = "0|1|4|5|3|2".split("|"),  
        _index = 0;  
    while (!![]) {  
        switch (+_array[_index++]){  
            case 0:  
                var str = formatStr;  
                continue;  
            case 1:  
                var Week = ['日', '一', '二', '三', '四', '五', '六'];  
                continue;  
            case 2:  
                return str;  
                continue;  
            case 3:  
                str = str.replace(/dd|DD/, this.getDate() > 9 ? this.getDate().toString() :  
                    '0' + this.getDate());  
                continue;  
            case 4:  
                str = str.replace(/yyyy|YYYY/, this.getFullYear());  
                continue;  
            case 5:  
                str = str.replace(/MM/, this.getMonth() + 1 > 9 ? (this.getMonth() + 1).  
                    toString() : '0' + (this.getMonth() + 1));  
                continue;  
        }  
        break;  
    }  
};  
console.log(new Date().format('yyyy-MM-dd'));
```

因为采用的是每一行语句对应一条 case 的方案,所以需要先获取每一行语句,处理方法与之前介绍的代码逐行混淆一致。遍历 FunctionExpression 节点,获取 path、node、body 即 BlockStatement。该节点的 body 属性是一个数组,通过 map 遍历数组,即可操作其中的每一行语句。代码如下:



视频讲解

```

    traverse(ast, {
      FunctionExpression(path){
        let blockStatement = path.node.body;
        let Statements = blockStatement.body.map(function(v, i){
          return {index: i, value: v};
        });
      });
    });

```

在流程平坦化的混淆中,要打乱原先的语句顺序,但是在执行时又要按原先的顺序执行,因此在打乱语句顺序之前,要对原先的语句顺序做映射。从上述代码中可以看出,采用 {index: i, value: v} 这种方式来做映射。index 是语句在 blockStatement.body 中的索引,也就是原先的顺序。value 是语句本身。有了对应的关系后,就可以方便地建立分发器,来控制代码在执行时跳转到正确的 case 语句块。

接着要打乱语句顺序,算法很简单,遍历数组每一个成员,每一次循环随机取出一个索引为 j 的成员,与索引为 i 的成员进行交换。代码如下:

```

//打乱语句顺序
let i = Statements.length;
while(i){
  let j = Math.floor(Math.random() * i--);
  [Statements[j], Statements[i]] = [Statements[i], Statements[j]];
}

```

接着构建 case 语句块,在 AST 中它的类型为 SwitchCase。下面介绍 SwitchCase 的结构:

```

{
  type: 'SwitchCase',
  test: { type: 'NumericLiteral', value: 5 },
  consequent: [
    Node { ... },
    { type: 'ContinueStatement', label: null }
  ]
}

```

test 是 case 后的值,consequent 是一个数组,存放 case 语句块中具体的语句,所以 case 语句块的生成很容易。遍历打乱顺序后的语句数组 Statements。SwitchCase 中的 test 用从 0 开始递增的 NumericLiteral,consequent 存放两条语句,一条是原始代码中的语句,另一条是 continue 语句。最后把包装好的 SwitchCase 放入 cases 数组中。实现代码如下:

```

let cases = [];
Statements.map(function(v, i){
  let switchCase = t.switchCase(t.numericLiteral(i), [v.value, t.continueStatement()]);
  cases.push(switchCase);
});

```

为了控制 switch 每一次都跳转到正确的 case 语句块上,需要构建分发器。分发器中的"0|1|4|5|3|2"来源于之前做的映射。调整上面用来生成 case 语句块的代码,改为如下形式:

```
let dispenserArr = [];  
let cases = [];  
Statements.map(function(v, i){  
  dispenserArr[v.index] = i;  
  let switchCase = t.switchCase(t.numericLiteral(i), [v.value, t.continueStatement()]);  
  cases.push(switchCase);  
});  
let dispenserStr = dispenserArr.join('|');
```

这段代码又定义了一个数组 dispenserArr,并且该数组在最后使用 join,以"|"作为连接符连接数组所有成员。可以看出,dispenserStr 就是分发器中的"0|1|4|5|3|2",而 014532 是代码执行的真实顺序,因此 dispenserArr 中的成员应当是 case 后面的值。假设 case 后面的值是 5,即 i 为 5,再取出当前语句的真实索引 v.index,假设它是 3,也就是原始代码中该语句是第 4 行语句(数组从 0 开始)。那么就把 dispenserArr 中索引为 3 的成员改为 5。对应上述代码中的 dispenserArr[v.index]=i,即可生成分发器中的字符串。

核心部分已经介绍完毕,接下来用 types 组件生成节点。完整的代码如下:

```
traverse(ast, {  
  FunctionExpression(path){  
    let blockStatement = path.node.body;  
    //逐行提取语句,按原先的语句顺序建立索引,包装成对象  
    let Statements = blockStatement.body.map(function(v, i){  
      return {index: i, value: v};  
    });  
    //打乱语句顺序  
    let i = Statements.length;  
    while(i){  
      let j = Math.floor(Math.random() * i--);  
      [Statements[j], Statements[i]] = [Statements[i], Statements[j]];  
    }  
    //构建分发器,创建 switchCase 数组  
    let dispenserArr = [];  
    let cases = [];  
    Statements.map(function(v, i){  
      dispenserArr[v.index] = i;  
      let switchCase = t.switchCase(t.numericLiteral(i), [v.value, t.continueStatement()]);  
      cases.push(switchCase);  
    });  
    let dispenserStr = dispenserArr.join('|');  
    //生成_array和_index标识符,利用 BabelAPI 保证不重名  
    let array = path.scope.generateUidIdentifier('array');  
    let index = path.scope.generateUidIdentifier('index');
```



```

//生成 '...'.split, 这是一个成员表达式
let callee = t.memberExpression(t.stringLiteral(dispenserStr), t.identifier('split'));
//生成 split('|')
let arrayInit = t.callExpression(callee, [t.stringLiteral('|')]);
//_array 和 _index 放入声明语句中, _index 初始化为 0
let varArray = t.variableDeclarator(array, arrayInit);
let varIndex = t.variableDeclarator(index, t.numericLiteral(0));
let dispenser = t.variableDeclaration('let', [varArray, varIndex]);
//生成 switch 中的表达式 + _array[_index++]
let updExp = t.updateExpression('++', index);
let memExp = t.memberExpression(array, updExp, true);
let discriminant = t.unaryExpression("!", memExp);
//构建整个 switch 语句
let switchSta = t.switchStatement(discriminant, cases);
//生成 while 循环中的条件 ![ ]
let unaExp = t.unaryExpression("!", t.arrayExpression());
unaExp = t.unaryExpression("!", unaExp);
//生成整个 while 循环
let whileSta = t.whileStatement(unaExp, t.blockStatement([switchSta, t.
breakStatement()]));
//用分发器和 while 循环来构建 blockStatement, 替换原有节点
path.get('body').replaceWith(t.blockStatement([dispenser, whileSta]));

}
});

/* 输出结果
Date.prototype.format = function (formatStr) {
  let _array = "3|0|4|1|5|2".split("|"),
  _index = 0;
  while (!![ ]) {
    switch ( + _array[_index++] ) {
      case 0:
        var Week = ['日', '一', '二', '三', '四', '五', '六'];
        continue;
      case 1:
        str = str.replace(/MM/, this.getMonth() + 1 > 9 ? (this.getMonth() + 1).
toString() : '0' + (this.getMonth() + 1));
        continue;
      case 2:
        return str;
        continue;
      case 3:
        var str = formatStr;
        continue;
      case 4:
        str = str.replace(/yyyy|YYYY/, this.getFullYear());
        continue;

```

```
        case 5:
            str = str.replace(/dd|DD/, this.getDate() > 9 ? this.getDate().toString()
: '0' + this.getDate());
            continue;
        }
        break;
    }
};
console.log(new Date().format('yyyy-MM-dd'));
*/
```

对上述代码做以下 3 点说明：

(1) 为了便于理解，代码没有使用其他混淆方案。但在实际使用中，该混淆方案必须和之前介绍的其他混淆方案配合使用，才能加强混淆效果。例如，case 后面跟的值可以用数值常量加密（在 JS 中 case 后面可以跟一个表达式，而不仅是数值或字符），分发器中的字符串可以用字符串加密，然后一起提取到大数组中。对于原始代码部分，可以进行生成花指令、逐行加密等各种操作。

(2) switch 中的表达式为 `+_array[_index++]`，最前面的加号代表强转成数值类型，因为分发器中它是字符串类型。虽然 JS 是一门弱类型的语言，解释器会在适当的时机完成类型的自动转换，但是在 case 中，全等（`===`）的匹配不会自动转换类型。

(3) 由于语句顺序被打乱，因此生成的语句顺序每一次都是不同的。在测试时，并不一定与上述顺序相同。

9.5.2 实现逗号表达式混淆

逗号表达式混淆的核心，是用逗号连接多个表达式。但是有一些语句在连接时，需要进行特别处理。

1. 声明语句之间的连接

示例代码如下

```
var a = 100;
var b = 200;
// var a = 100, b = 200;
// var a = 100, var b = 200; 不能这样操作
```

可以看出，如果要连接两个声明语句，需要取出 `VariableDeclaration` 中的 `declarations` 数组，该数组里是声明语句中定义的变量，然后将它处理成一条声明语句。这里最方便的处理方式是把所有的标识符声明都提取到参数中。

2. 普通语句与 return 语句连接

示例代码如下：

```
function test(a){
    a = a + 100;
```



视频讲解

```

    return a;
  }
  /*
    function test(a){
      return a = a + 100,
      a;
      // a = a + 100, return a; 不能这样操作
    }
  */

```

可以看出,普通语句与 return 语句连接时,需要提取出 return 语句的 argument 部分,然后重新构建 return 节点,把整个逗号表达式作为 argument 部分。

接着介绍如何把函数中所有声明的变量提取到参数列表中,实现代码如下:

```

traverse(ast, {
  FunctionExpression(path){
    let blockStatement = path.node.body;
    let blockStatementLength = blockStatement.body.length;
    if(blockStatementLength < 2) return;
    //把所有声明的变量提取到参数列表中
    path.traverse({
      VariableDeclaration(p){
        declarations = p.node.declarations;
        let statements = [];
        declarations.map(function(v){
          path.node.params.push(v.id);
          v.init && statements.push(t.assignmentExpression('=', v.id, v.init));
        });
        p.replaceInline(statements);
      }
    });
  }
});

```

遍历 FunctionExpression 节点,取出 blockStatement.body 节点,该节点里存放函数体中所有的语句。如果语句少于两条,就不做任何处理。接着,遍历当前函数下所有的 VariableDeclaration 节点,其中 declarations 数组为声明的具体变量。然后通过 map 遍历整个数组,通过 path.node.params.push(v.id) 将变量提取到函数的参数列表中。要考虑两种情况:变量没有初始化,那么它不加入到 statements 数组中,最后替换节点时相当于被移除;变量初始化,那么就将 VariableDeclarator 改为赋值语句。

有时候会在函数体中的语句外面包裹一层 ExpressionStatement 节点,这会影响语句类型的判断。所以先把这类语句外层的 ExpressionStatement 节点去掉,代码如下:

```

let firstSta = blockStatement.body[0], i = 1;
while(i < blockStatementLength){
  let tempSta = blockStatement.body[i++];
  t.isExpressionStatement(tempSta) ?
    secondSta = tempSta.expression : secondSta = tempSta;
}

```

首先取出函数体中的第一条语句,并且定义一个初始值为1的计数器*i*。接着对函数体中的其他语句进行循环,取出来的语句先赋值给 *tempSta*,然后判断是否为 *ExpressionStatement* 节点。如果是,就取出 *expression* 属性赋值给 *secondSta*,否则直接赋值给 *secondSta*。

然后要把这两个语句改为逗号表达式,可以使用 *toSequenceExpression* 来完成。对于不同的语句,需要使用不同的处理方案。本节只处理其中的赋值语句和函数调用语句,返回语句也需要处理,如下所示:

```
//处理返回语句
if(t.isReturnStatement(secondSta)){
    firstSta = t.returnStatement(
        t.toSequenceExpression([firstSta, secondSta.argument]));
}
//处理赋值语句
}else if(t.isAssignmentExpression(secondSta)){
    secondSta.right = t.toSequenceExpression([firstSta, secondSta.right]);
    firstSta = secondSta;
}else{
    firstSta = t.toSequenceExpression([firstSta, secondSta]);
}
```

首先介绍这段代码中的赋值语句的处理方法。判断语句如果是赋值表达式,就取出其中的 *right* 节点,与 *firstSta* 组成逗号表达式后,再替换原有的 *right* 节点,最后把 *secondSta* 赋值给 *firstSta*,进行后续的语句处理。接着介绍返回语句,取出其中的 *argument* 节点,与 *firstSta* 组成逗号表达式后,重新生成一个 *returnStatement* 节点。到这一步就可以跳出循环,因为后续即使再有语句,也不会被执行到。如果既不是返回语句,也不是赋值语句,那么就直接将 *firstSta* 与 *secondSta* 组成逗号表达式,这是一种最没有混淆力度的组成方式。

最后再处理函数调用表达式。在原始代码中, *str.replace(...)* 可以处理成 (*firstSta*, *str*).*replace(...)* 或者 (*firstSta*, *str.replace(...)*), 本节来实现第一种情况,如下所示:

```
if(t.isCallExpression(secondSta.right)){
    let callee = secondSta.right.callee;
    callee.object = t.toSequenceExpression([firstSta, callee.object]);
    firstSta = secondSta;
}
```

这段代码中,赋值语句的右边是函数调用表达式。取出函数名 *callee*, *str.replace(...)* 的函数名为 *str.replace*。这是一个 *MemberExpression* 节点。接着取出 *object* 部分,即将 *str* 与 *firstSta* 组成逗号表达式,再替换掉原先的 *object* 节点。

下面展示完整的处理代码:

```
traverse(ast, {
    FunctionExpression(path){
        let blockStatement = path.node.body;
        let blockStatementLength = blockStatement.body.length;
        if(blockStatementLength < 2) return;
```

```

//把所有声明的变量提取到参数列表中
path.traverse({
  VariableDeclaration(p){
    declarations = p.node.declarations;
    let statements = [];
    declarations.map(function(v){
      path.node.params.push(v.id);
      v.init && statements.push(t.assignmentExpression('=', v.id, v.init));
    });
    p.replaceInline(statements);
  }
});
//处理赋值语句、返回语句和函数调用语句
let firstSta = blockStatement.body[0], i = 1;
while(i < blockStatementLength){
  let tempSta = blockStatement.body[i++];
  t.isExpressionStatement(tempSta) ?
    secondSta = tempSta.expression : secondSta = tempSta;
  //处理返回语句
  if(t.isReturnStatement(secondSta)){
    firstSta = t.returnStatement(
      t.toSequenceExpression([firstSta, secondSta.argument]));
  //处理赋值语句
  }else if(t.isAssignmentExpression(secondSta)){
    if(t.isCallExpression(secondSta.right)){
      let callee = secondSta.right.callee;
      callee.object = t.toSequenceExpression([firstSta, callee.object]);
      firstSta = secondSta;
    }else{
      secondSta.right = t.toSequenceExpression([firstSta, secondSta.right]);
      firstSta = secondSta;
    }
  }else{
    firstSta = t.toSequenceExpression([firstSta, secondSta]);
  }
}
path.get('body').replaceWith(t.blockStatement([firstSta]));
});

/* 输出结果
Date.prototype.format = function (formatStr, str, Week) {
  return str = (str = (str = (Week = (str = formatStr, ['日', '一', '二', '三', '四',
'五', '六'], str).replace(/yyyy|YYYY/, this.getFullYear()), str).replace(/MM/, this.getMonth() + 1 > 9 ? (this.getMonth() + 1).toString() : '0' + (this.getMonth() + 1)), str).replace(/dd|DD/, this.getDate() > 9 ? this.getDate().toString() : '0' + this.getDate()), str;
);
  console.log(new Date().format('yyyy-MM-dd'));
}
*/

```

这段代码只适用于原始代码的案例,实际应用中,还需要考虑更多情况。可以看出,逗号表达式混淆的实现是比较复杂的,但是逗号表达式的还原很容易处理。后续章节会给出相应的还原方案。

9.5 小结

本章介绍了很多混淆的实现方案,这些方案之间需要配合使用才能达到更好的混淆效果。但是在配合使用的过程中,有些方案需要注意顺序,如代码逐行加密,由于会把代码中的标识符一起加密,所以需要放在标识符混淆之后;有些方案不需要注意顺序,如数值常量混淆。在代码执行逻辑的混淆中,流程平坦化的实现方案也有很多种,大体原理差不多。

此外,本章不少小节只写了 FunctionExpression 的情况,FunctionDeclaration 的处理方法也是一样的。

9.6 习题

1. 使用 Babel 的 API,将 `c(d)` 转换为以下形式:

```
function xxx(a, b){
    return a(b);
}
xxx(c, d);
```

2. 本章介绍的字符串加密方案使用 Base64 编码,在解密时使用浏览器自带的 `atob` 方法。请实现一个 Base64 解码函数,来替换方案中的 `atob` 方法。
3. 完善本章中逗号表达式混淆的实现代码,使它能够更好地处理以下代码中的 `test`。

```
var obj = {
    name: 'xiaojianbang',
    add: function (a, b){
        return a + b;
    }
};
function sub(a, b) {
    return a - b;
}
function test() {
    var a = 1000;
    var b = sub(a, 3000) + 1;
    var c = b + obj.add(b, 2000);
    return c + obj.name;
}
//将 test 处理成以下形式
function test(a, b, c) {
    return c = (b = (a = 1000, sub)(a, 3000) + 1, b + (0, obj).add(b, 2000)), c + (0, obj).name;
}
```

第10章

AST自动化JS还原方案



本章着重讲述 JavaScript(以下简称 JS)还原方案的代码实现。本章只讲述一部分的代码还原方案,更多还原方案将在下一章的实战案例中介绍。同时,本章还会介绍 Chrome 扩展、JS Hook 等前沿技术。

10.1 常用还原方案

10.1.1 还原数值常量加密

在 9.4 节混淆后的代码中,可以看到有以下代码。

```
...
eval(String.fromCharCode(425871 ^ 425977, 118436 ^ 118469, ... ));
...
```

其中 `String.fromCharCode` 中的参数,就是之前做的数值常量加密混淆,那么如何还原呢?思路很简单,遍历 `BinaryExpression` 节点,取出 `left` 和 `right`。当 `left` 和 `right` 节点类型都为 `NumericLiteral` 时,才是要还原的节点。接着调用 `path.evaluate` 方法来计算节点的值,最后构造新的节点替换回去。实现还原的代码如下:

```
traverse(ast, {
  BinaryExpression(path){
    let left = path.node.left;
    let right = path.node.right;
    if(t.isNumericLiteral(left) && t.isNumericLiteral(right)){
      let {confident, value} = path.evaluate();
      confident && path.replaceWith(t.valueToNode(value));
    }
  }
});
```

```
    }  
  }  
});  
  
/*  
...  
eval(String.fromCharCode(118, 97, 114, 32, 79, 79, 79, 79, 79, 48, 32, 61, 32, 79, 79, 79, 79,  
79, 111, 59));  
...  
*/
```

path.evaluate 方法用来计算节点的值,返回一个对象。返回的对象中的 confident 是一个布尔值,它代表节点是否可计算出结果。

如果可以计算出结果,那么返回的对象中的 value 就存放该值。

10.1.2 还原代码加密与 ASCII 码混淆

代码经过 10.1.1 节处理后,可以看到代码中有一处进行了加密,还有一处进行了 ASCII 码混淆。具体代码如下:

```
eval(String.fromCharCode(118, 97, 114, 32, 79, 79, 79, 79, 79, 48, 32, 61, 32, 79, 79, 79, 79,  
79, 111, 59));  
...  
eval(atob("T09PT08wID0gT09PT08wW2F0b2IoT09PT09PWzEwXSldKC95eXl5fF1ZWVkvLC  
B0aGlzW2F0b2IoT09PT09PWzExXSldKkCkP0w == "));
```

现在要做的是将它们解密成明文代码。如果对代码进行加密或者 ASCII 码混淆,那么需要配合 eval 或者 Function 使用。Function 主要用来创建一个函数,所以代码中使用的是 eval。那么思路就来了,既然这两种混淆方式都需要使用 eval,那就可以一起处理。遍历 CallExpression 节点,判断该节点下的属性 callee.name 是否等于 eval,如果相等,那么就是需要处理的地方。eval 是用来执行字符串形式的代码的,它的参数是字符串,如果不是字符串,说明还需要经过计算才能得到字符串。因此,如果 eval 中的参数是 StringLiteral 节点,那么直接使用 types 组件的 identifier 让参数变成代码,然后替换整个 CallExpression 节点;如果不是 StringLiteral 节点,那么就用 generator 组件,将 CallExpression 的 arguments 节点转为字符串,接着使用 eval 来执行字符串代码,得到结果。实现还原的代码如下:

```
function atob(e) {  
  ...  
}  
traverse(ast, {  
  CallExpression(path) {  
    if (path.node.callee.name !== 'eval') return;  
    let arguments = path.node.arguments;  
    let code = generator(arguments[0]).code;  
    if (t.isStringLiteral(arguments)) {
```



```

        path.replaceWith(t.identifier(code));
    }else{
        path.replaceWith(t.identifier(eval(code)));
    }
    });
});

/*
...
var 000000 = 00000o;; //ASCIIEncrypt
...
000000 = 000000[ atob(000000[10])] (/yyyy| YYYY/, this[ atob(000000[11]])());; //
Base64Encrypt
*/

```

注意,需要找到相应的解密函数,并且在定义时,名字要一致。例如上述代码中的 atob,它是浏览器中的 Base64 解密函数,并不能在文件中找到它的定义。因此需要在代码中自己实现,并且名字需要为 atob。

10.1.3 还原 unicode 与十六进制字符串

在前面章节中介绍过,JS 中的字符串可以表示成 unicode 形式或者十六进制形式,并且 JS 中的标识符也可以表示成 unicode 形式。在 Babel 中还原它们很容易对于标识符表示成 unicode 形式的情况,首先用 parser 组件解析,再用 generator 组件转成代码即可,generator 不需要指定任何选项。但如果是字符串表示成 unicode 形式或者十六进制形式的情况,就需要额外指定一些选项。以 6.1.3 节处理后的代码为例,实现还原的代码如下:

```

/* 处理前的代码
Date.prototype.\u0066\u006f\u0072\u006d\u0061\u0074 = function(formatStr) {
    var \u0073\u0074\u0072 =
    \u0066\u006f\u0072\u006d\u0061\u0074\u0053\u0074\u0072;
    var Week = [\u065e5', '\u4e00', '\u4e8c', '\u4e09', '\u56db', '\u4e94',
    '\u516d'];
    ...
}
console.log( new \u0077\u0069\u006e\u0064\u006f\u0077[ '\u0044\u0061\u0074\u0065']()
['format'](' \x79\x79\x79\x79\x2d\x4d\x4d\x2d\x64\x64' ));
*/
const jscode = fs.readFileSync("./demo.js", {
    encoding: "utf-8"
});
let ast = parser.parse(jscode);
let code = generator(ast, {minified: true, jscOption: {minimal: true}}).code;
ast = parser.parse(code);
code = generator(ast).code;
console.log(code);

```

```
/* 处理后的代码
Date.prototype.format = function (formatStr) {
    var str = formatStr;
    var Week = ["日", "一", "二", "三", "四", "五", "六"];
    ...
};
console.log(new window["Date"]()[ "format" ]( "yyyy - MM - dd" ));
*/
```

generator 的第二个参数有很多选项,可以在 vscode 中,按 Ctrl 键,然后单击 minified 跳转到 ts 文件中查看这些选项,里面还有很多注释来解释选项的作用。第一次使用 generator 时,就已经把代码还原了。第二次的 parse 和 generator 只是为了把代码格式化。

10.1.4 还原逗号表达式混淆

逗号表达式的还原相对于混淆的实现来说,要容易得多。以 6.3.2 节中的代码为例,代码如下:

```
function test2(a, b, c, d, e, f){
    return f = (e = (d = (c = (b = (a = 1000, a + 50, b + 60, c + 70, a + 2000), d + 80, b
    + 3000), e + 90, c + 4000), f + 100, d + 5000), e + 6000);
}
console.log( test2() );
```

将这段代码中的(a=1000,a+50,b+60,c+70,a+2000) 放在 AST Explorer 网站中解析,可以看到如下结构:

```
{  "type": "SequenceExpression",
  ...
  "expressions": [
    { "type": "AssignmentExpression" ... },
    { "type": "BinaryExpression" ... },
    { "type": "BinaryExpression" ... },
    { "type": "BinaryExpression" ... },
    {
      "type": "BinaryExpression",
      ...
      "left": {
        "type": "Identifier",
        ...
        "name": "a"
      },
      "operator": "+",
      "right": {
        "type": "NumericLiteral",
        ...
      }
    }
  ]
}
```

```
        "value": 2000
      }
    }
  ],
  ... }
```

可以看出,逗号表达式在 AST 的类型为 `SequenceExpression`,其中 `expressions` 节点就是逗号运算符连接的每一个表达式,`expressions` 数组中的最后一个成员就是逗号表达式整体返回的结果。实现还原的代码如下:

```
traverse(ast, {
  SequenceExpression: {
    exit(path) {
      let expressions = path.node.expressions;
      let finalExpression = expressions.pop();
      let statement = path.getStatementParent();
      expressions.map(function(v) {
        statement.insertBefore(t.ExpressionStatement(v));
      });
      path.replaceInline(finalExpression);
    }
  }
});
/*
function test2(a, b, c, d, e, f) {
  a = 1000;
  a + 50;
  b + 60;
  c + 70;
  b = a + 2000;
  d + 80;
  c = b + 3000;
  e + 90;
  d = c + 4000;
  f + 100;
  e = d + 5000;
  return f = e + 6000;
}
console.log(test2());
*/
```

这段代码遍历 `SequenceExpression` 节点,取出 `expressions` 数组,将该数组的最后一个成员赋值给 `finalExpression`,作为整个逗号表达式节点的返回结果。接着调用 `path.getStatementParent` 获取最近的父语句节点,在这里是 `ReturnStatement` 节点。然后把数组中的其他成员加入到该节点的前面,最后用 `finalExpression` 替换整个逗号表达式节点。推荐使用 `exit` 时处理,因为更符合手动还原时的逻辑。

也可以用上述代码处理 9.5.2 节中的逗号表达式混淆代码,最终输出代码如下:

```
/*
Date.prototype.format = function (formatStr, str, Week) {
    return str = (str = (str = (Week = (str = formatStr, ['日', '一', '二', '三', '四', '五', '六'], str).replace(/yyyy|YYYY/, this.getFullYear()), str).replace(/MM/, this.getMonth() + 1 > 9 ? (this.getMonth() + 1).toString() : '0' + (this.getMonth() + 1)), str).replace(/dd|DD/, this.getDate() > 9 ? this.getDate().toString() : '0' + this.getDate()), str;
    };
console.log(new Date().format('yyyy-MM-dd'));
*/

Date.prototype.format = function (formatStr, str, Week) {
    str = formatStr;
    Week = ['日', '一', '二', '三', '四', '五', '六'];
    str = str.replace(/yyyy|YYYY/, this.getFullYear());
    str = str.replace(/MM/, this.getMonth() + 1 > 9 ? (this.getMonth() + 1).toString() : '0' + (this.getMonth() + 1));
    str = str.replace(/dd|DD/, this.getDate() > 9 ? this.getDate().toString() : '0' + this.getDate());
    return str;
};
console.log(new Date().format('yyyy-MM-dd'));
```

10.2 Chrome 拓展开发入门

10.2.1 Chrome 拓展程序

Chrome 拓展程序可以扩展 Chrome 浏览器的功能,让开发者可以定制自己的浏览器。它工作在浏览器层面,主要使用 HTML 和 JS 进行开发,通过调用 Chrome 提供的 API 来实现下列 7 种功能:

- (1) 标签控制。
- (2) 书签控制。
- (3) 窗口控制。
- (4) 下载控制。
- (5) 请求控制。
- (6) 事件监听。
- (7) 网页修改。

例如,可以使用它来识别一个网站中的广告代码,当广告出现在网页上时,它可以自动删除。

如图 10-1 所示,如果要安装和管理 Chrome 拓展程序,可以在地址栏中输入“chrome://extensions”进行访问。选择“开发者模式”选项即可以文件夹的形式直接加载插件,否则只能安装 crx 格式的文件。

开发 Chrome 插件并没有太高的门槛,只需要保证在目录下有一个叫作 manifest.json 的文件。manifest.json 用来配置所有和插件相关的参数,虽然存在很多参数,但只有

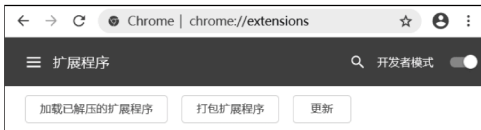


图 10-1 管理安装 Chrome 拓展程序

manifest_version、name 和 version 这三个参数是必不可少的。下面介绍一些常见的 Chrome 配置项。

```
{
  "name": "插件示例",           //拓展插件名称
  "version": "2.0.0",           //拓展插件版本
  "description": "插件示例",     //拓展插件描述
  //请求的权限
  "permissions": [
    "storage",                  //本地存储权限
    "activeTab",                //与当前页面交互的 API 权限
    "tabs",                     //管理 Chrome 浏览器标签栏权限
    "debugger",                 //启动 debugger 工具权限
  ],
  //向页面注入的 JS 脚本配置
  "content_scripts": [
    {
      //匹配网址的正则表达式列表
      "matches": ["http://*/*", "https://*/*"],
      //JS 脚本注入的时间,此处是 DOM 创建完毕,但子资源尚未加载前
      "run_at": "document_end",
      //需要注入的 JS 脚本文件列表
      "js": ["inject.js"],
      //是否运行在页面所有的 iframe 中
      "all_frames": true
    }
  ],
  //常驻后台的脚本
  "background": {
    "scripts": ["background.js"], //在关闭浏览器之前一直执行的脚本
  },
  "manifest_version": 2 //manifest 文件版本号
}
```

从上述配置项可知,content_script.js 是 Chrome 拓展程序向页面注入脚本的一种形式,run_at 有以下三种注入时机:

- (1) document_start: 所有 CSS 加载完毕,但是 DOM 尚未创建时。
- (2) document_end: DOM 创建完毕,但图片和 frame 等子资源尚未加载时。
- (3) document_idle: document_end 之后,页面完全载入之前。

如果想要去除一个页面中的广告,需要选择在 DOM 创建完毕之后注入。

background 是常驻后台的一个页面,它的生命周期是 Chrome 拓展程序中所有类型页面中最长的,只有在浏览器关闭时才会停止运行,所以通常需要把全局且需要长期运行的代码放在这里。特别说明的是,虽然可以通过 `chrome-extension://xxx/background.html` 直接打开后台页,但是打开的后台页和真正一直在后台运行的页面不是同一个页面,换句话说,尽管可以打开无数个 `background.html`,但是真正在后台常驻的只有一个页面,而且永远看不到该页面的界面,只能调试它的代码。

10.2.2 Chrome 拓展开发之去除广告插件

去除广告的原理很简单,首先定位到元素面板的广告节点,然后用 JS 方法 `remove` 去除。这里使用 Chrome 插件来实现,定位的函数是 `document.querySelectorAll`,获取到的是 DOM 对象,因而可以直接使用它下属的 `remove` 方法去除。定位 CSS 选择器需要根据具体的网站结构来编写。注入时机是在 DOM 渲染之后,也就是 `document.end`。用去除百度搜索引擎的搜索结果中的广告为例,需要在 `manifest` 中编写以下 `content_scripts`:

```
"content_scripts": [{
  "matches": ["http:// * / * ", "https:// * / * "],
  "run_at": "document_end",
  "js": ["inject.js"],
  "all_frames": true
}]
```

`inject.js` 需要根据页面广告的 CSS 编写:

```
document.querySelectorAll(".EGZQLh.eWxNhq.RrDTyC.KNVPwj.EC_ppim_new_gap_bottom").forEach(
(function(a){a.remove()}))
```

插件安装后,会把百度搜索结果的广告自动去除掉。

10.3 JS Hook

10.3.1 JS Hook 原理与作用

假设有以下这段 JS 代码:

```
function test(aa,bb){
  cc = aa + bb;
  return cc;}test(100,200);
```

在 `console` 中可以修改这个函数,例如让它打印各个参数的输出结果,这就是一个 JS Hook:

```
var _test = test; test = function(aa, bb) {  
    console.log(aa);  
    console.log(bb);  
    var result = _test(aa, bb);  
    console(result);  
}
```

在函数调用之后,页面加载完毕才注入函数。但实际上可以在调用的地方设置断点,运行时注入 JS Hook,在函数被调用之前就进行修改。JS Hook 看起来很简单,功能却很强大,例如修改一些系统函数,如 debugger、Function、eval 等,这些都是和反调试相关的。数据大多都是 JSON,这是一种轻量级的数据交换格式所以可以使用 HookJSON、stringfy 进行操作,就可以在 Hook 后直接从堆栈中找到调用函数了。

无论如何进行 Hook,都要把代码注入到网页的环境里,注入时机的选择很重要。可以选择设置断点注入,但更为便捷的方法是编写 Chrome 拓展插件,插件可以在网页运行之前对系统函数进行 Hook,一些网页的反调试就可以直接解决了。

10.3.2 JS Hook 对象属性

要想对对象属性进行 Hook,需要用到 Object.defineProperty 或者 Object.defineProperty 方法,它们可以直接在一个对象上修改原有属性或者定义新的属性,如下所示:

```
//Object.defineProperty(obj, props)  
/**  
obj: 将要被添加属性或修改属性的对象  
props: 该对象的一个或多个键值对,定义了将要为对象添加或修改的属性的具体配置  
**/  
var obj = new Object();  
Object.defineProperty(obj, {  
    name: {  
        value: 'JS Hook',  
        configurable: false,  
        writable: true,  
        enumerable: true  
    },  
    age: {  
        value: 18,  
        configurable: true  
    }  
}))
```

对象生成之后就可以进行 Hook 了。例如以下 Hook 示例,它会在每次设置对象参数时进行 debugger。

```
//Object.defineProperty(obj, prop, descriptor)  
/**  
obj: 需要被操作的目标对象
```

```
prop: 目标对象需要定义或修改的属性的名称
descriptor: 将被定义或修改的属性的描述符
** /
var obj = new Object();
Object.defineProperty(obj, 'name', {
  set:function(x){debugger;return x;}//被赋值后断下来})
```

对于内置对象属性的 Hook,需要在文档加载之前进行 Hook,这时就需要用到 Chrome 拓展插件。例如 document.cookie 中的各种 cookies,如果想知道 cookies 是从哪里生成的,使用 Chrome 拓展很方便,使用断点调试可能比较麻烦,可以在 JS 文件头部设置断点,在 console 里边输入如下代码。

```
Object.defineProperty(document,"cookie",
{set:function(x){console.log(x);return x;}})
```

然后运行代码,就会看到 cookie 在控制台被打印。要查看 cookie 在哪里被定义,可以写入 debugger。要查看特定的 cookie,可以使用 indexOf 指定参数名字。

10.3.3 JS 自动注入 Hook

接下来,可以编写一个 Chrome 拓展插件。这里给出 content_scripts 的脚本文件,让它在页面 DOM 渲染之前自动注入 JS 脚本文件,从而达到反调试的效果。

在页面创建一个 script 节点,里边放入注入的代码。以下代码实现了 Hook Cookies。

```
(function(){
  'use strict';
  var s = document.createElement('script');

  //Hook Cookies
  s.innerHTML=`var cookie_cache = document.cookie;
Object.defineProperty(document,'cookie',{
  get:function(){return cookie_cache;},
  set:function(val){
console.log("setting cookie:",val);
//debugger;
var cookie = val.split(";")[0];
var ncookie = cookie.split("=");
var flag = false;
var cache = cookie_cache.split(";");
cache = cache.map(function(a){
if (a.split("=")[0] == ncookie[0]){
  flag = true;
  return cookie;}
return a;})
cookie_cache = cache.join(";");
if(!flag){`
```



```
cookie_cache += cookie + ";";}  
this._value = val;  
return cookie_cache;}});})
```

10.4 DOM 对象的 Hook

10.4.1 Script 自动加载

以 Hook Script 脚本请求为例,生成一个 script 元素,指定好 script 的 src 属性,当把这个元素加入到 HTML 中时,会发起一个请求,并且这个请求可以跨域。

用一个简单的例子证明这一点,首先在本地创建一个 HTTP 接口,在访问这个接口时会返回自定义的 JS 脚本,如下所示:

```
from flask import Flask  
app = Flask(__name__)  
@app.route('/')  
def index():  
    res = ""  
    function test(){  
        alert("调用 HTTP 服务!")  
    };  
    test();  
    ""  
    return res  
if __name__ == '__main__':  
    app.run(port = 8080)
```

在任意一个页面中注入 src 为 `http://127.0.0.1:8080` 的 script 节点,如下所示:

```
var myScript = document.createElement("script");  
myScript .src = "http://127.0.0.1:8080";  
document. body. appendChild(myScript);
```

该页面上会弹出内容为“调用 HTTP 服务”的窗口,可以证明 script 节点的 src 会被自动请求。

10.4.2 Hook DOM

示例代码如下:

```
<!DOCTYPE html>  
<  
html lang = "en">  
< head>  
    < meta charset = "UTF - 8">
```

```
</head>
<body>
</body>
<script type="text/javascript">
  var s=document.createElement('script');
  s.src="https://www.baidu.com";
  document.documentElement.appendChild(s);

  (function()
  {
    var s=document.createElement('script');
    s.src="https://i.qq.com";
    document.documentElement.appendChild(s);
  }());
</script>
</html>
```

在面对上述代码时,如何定位到 `appendChild` 使用的地方?既要定位到外部的 `appendChild`,也要能够定位到匿名函数中的 `appendChild`。换言之,假如现在的代码有几千行甚至上百万行,还经过了混淆,要想快速定位到 `appendChild` 该怎么做?这时,就需要对 `appendChild` 进行 Hook。

`appendChild` 并不是一个全局函数,如果要进行 Hook,就需要连带前边的对象:

```
_document.documentElement.appendChild = document.documentElement.appendChild
```

这样的 Hook 是有局限性的,如果 JS 脚本以其他方式添加进页面,如获取 `body`,然后把它作为 `body` 的子元素,当前的 Hook 就会失效,所以当前对象的 Hook 最好从原型链的角度出发。

`appendChild` 是 DOM 对象的方法,可以直接在 Console 面板中查看该 DOM 对象的原型链,发现它的 `__proto__` 是一个 `HTMLScriptElement`,它的内部包含了许多属性,这些属性都是可以自由赋值的,如图 10-2 所示。

直接在当前属性中寻找 `appendChild` 是找不到的,在 `__proto__` 属性回溯的过程中,会发现 `appendChild` 存在于 Node 对象的属性中,如图 10-3 所示。

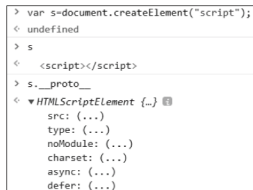


图 10-2 HTMLScriptElement 原型链

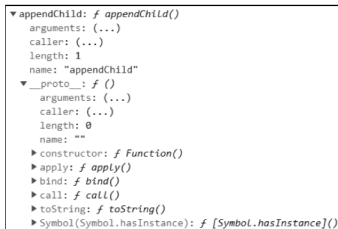


图 10-3 appendChild 属性

要想对 `appendChild` 进行 Hook, 只需要 Hook `Node.prototype.appendChild` 即可, 代码如下:

```
//Hook appendChild
var _appendChild = Node.prototype.appendChild;
Node.prototype.appendChild = function(){
    console.log("Hook appendChild");
    return _appendChild();
};
```

10.4.3 JS Proxy

Proxy 代理可以帮助完成很多事情, 如数据的处理、对构造函数的处理和对数据的验证。它在访问对象前添加了一层拦截, 进行过滤操作, 其中的过滤可自行定义, 例如:

```
let p = new Proxy(target, handler);
```

(1) `target`: 需要使用 Proxy 包装的目标对象(可以是任何类型的对象, 包括原生数组、函数或另一个代理)。

(2) `handler`: 一个对象, 其属性是当执行一个操作时定义代理的行为的函数(可以理解为某种触发器)。

例如前面几节介绍的 `appendChild` 的 Hook, 可以改为如下 Proxy 的形式:

```
//Proxy appendChild
Node.prototype.appendChild = new Proxy(
    Node.prototype.appendChild, {
    apply: function(trapTarget, thisArg, argumentList){
        var retval = Reflect.apply(trapTarget, thisArg, argumentList);
        console.log("appendChild retval:", retval);
        return retval;
    }
});
```

10.5 原型链

JS 中所有的对象都是 `Object` 的实例。在创建对象时, 会有一些预定义的属性。其中, 定义函数时, 预定义属性是 `prototype`, `prototype` 是一个普通的对象。定义普通对象时, 会生成一个 `__proto__`, `__proto__` 指向这个对象的构造函数的 `prototype`。

每一个 JS 对象(`null` 除外)在创建时都会与之关联另一个对象, 这个对象就是原型, 每一个对象都会从原型继承属性。

一连串的对象依托 `__proto__` 和 `prototype` 连接, 就形成了原型链, 又因为 JS 中所有的

对象都是 Object 的实例,所以原型链的终点是 Object。

要进一步理解原型链,需要掌握以下三个属性。

1. __proto__ 属性

JS 中的每一个对象都拥有 __proto__ 属性,这个属性会指向自身的原型对象。在访问一个对象属性时,如果这个对象的内部不存在这个属性,就会经过 __proto__ 属性到原型对象里寻找,如果上一级的原型对象里也不存在这个属性,就会继续沿着上一级的 __proto__ 属性回溯,一直到原型链的终点 Object,在此之后就是 null 了。

在 JS 中定义了一个对象之后,并没有为它设置字符串方法或者函数方法,却可以直接使用,是因为它通过 __proto__ 属性从原型对象继承而来。

2. prototype 属性

prototype 属性是 JS 中函数独有的,函数也可以看作对象,因此它也拥有 __proto__ 属性。prototype 属性会指向该函数所创建的示例的原型对象,也就是说,一个函数实例的 __proto__ 就是该函数的 prototype,如下所示:

```
function Test() {}  
var test = new Test();  
console.log(test.__proto__ === Test.prototype); // true
```

3. constructor 属性

每个对象都可以找到其对应的 constructor 属性,创建对象的前提是要拥有 constructor。它可以是显式定义的,也可以是通过 __proto__ 从原型对象继承而来。constructor 属性指向该对象的构造函数,所有函数和对象最终都是由 Function 构造函数得来的,所以 constructor 属性的终点是 Function 构造函数。

在每个函数创建时,JS 会同时创建一个该函数对应的 prototype 对象,而函数创建的实例对象, __proto__ === 函数, prototype, 函数, prototype, constructor === 函数本身, 所以通过函数创建的对象即使没有 constructor 属性,它也能通过 __proto__ 找到对应的 constructor,所以任何对象最终都可以找到其构造函数(null 对象除外)。

```
function Test() {}  
console.log(Test === Test.prototype.constructor); // true
```

通过一个简单的例子来理解原型链:

```
function Test() {}  
//原型属性  
Test.prototype.name = 'hello'  
var newTest = new Test()  
//实例属性  
newTest.name = 'world'  
console.log(newTest.name)//world
```

这段代码在原型属性和实例属性中都有一个名为 name 的属性,但是最后输出的是实例属性的值。读取一个属性时,如果在实例属性上找到,就读取它,不会管原型属性上是否还有相同的属性,这就是属性屏蔽,即当实例属性和原型属性拥有相同名字时,实例属性会屏蔽原型属性但它不会修改原型属性上的值;如果在实例属性上没有找到,就会在实例的原型上去找,如果原型上也没有找到,就继续到该原型的原型上去找,直到尽头。

10.6 XHR Hook

XHR Hook 就是本书在第 2 章中讲过的 Ajax-Hook,自定义 XMLHttpRequest 中的方法和属性,覆盖全局的 XMLHttpRequest。

10.7 JS Hook 的检测

可以应用 JS 代码格式化的检测方法到 JS Hook 的检测。核心思想是一样的,代码格式化和 Hook 都会改变原始函数的代码结构。如何检测 JS 代码格式化,在本书的第 6 章中有详细描述。

最常见的 JS Hook 就是对比关键函数前后的 JS 文本,如果不一致,就进入循环 debugger。

```
function test(x,y){
    z = x + y;
    return z;}
setInterval(function(){test + "" == "function test(x,y){z = x + y;return z;}"}
? console.log("未修改")
:setInterval(function(){eval("debugger")},1000);},1000);
test(100,200);
```

如果要过滤掉这种 Hook 检测,可以编写如下代码:

```
Function.prototype.toString = function(){
    return "function test(x,y){z = x + y;return z;}";}
```

还可以检测 setInterval 中的参数代码,如果代码中有 debugger,就进行过滤,代码如下:

```
var _setInterval = setInterval;setInterval = function(a,b){
    if(a.toString().indexOf("debugger")!= - 1{
        return null;
    }
    _setInterval(a,b);}
```

10.8 小结

本章讲述了部分代码还原方案,更多还原方案和实战应用将在下一章介绍。本章还介绍了 Chrome 扩展、JS Hook 等技术,可以将其中的技术结合起来,编写自定义的 Chrome 拓展工具。

10.9 习题

1. 编写一个 Chrome 拓展程序,自动输出网页的 Cookie。
2. 简述 JS Proxy 的优点。
3. 什么是 JS 中的原型链?
4. XHR Hook 是用来做什么的?

第11章

AST还原JS实战

本章以一个实际案例,来演示 AST 在还原 JS 混淆上的应用。首先介绍如何分析网站使用的混淆手段,这是用 AST 还原代码的前提。其次介绍如何巧妙使用 Babel 中的 API 来还原这些混淆。最后介绍如何利用还原后的文件来进行协议逆向。还原后的 JS 文件并不是只能静态分析,如果还原做得优秀,还可以替换到原网站中进行动态调试。



视频讲解

11.1 分析网站使用的混淆手段

11.1.1 协议分析

AST 只是用来处理 JS 代码的,所以对于协议的抓包以及对应加密参数的分析,是开发者必须去做的。某网站在输入用户名和密码后,单击“登录”选项,会弹出验证码图片,如图 11-1 所示。单击任意两个选项,网站会验证答案是否正确。本章使用 Chrome 浏览器自带的开发者工具抓包,去分析抓到的数据包。

这里只分析关键的数据包。获取验证码的请求为 /yzmtest/get.php?t=1596249640810,这是一个 POST 请求,提交的数据为 clientid=gl03ushm0yw&username=e511111111,其中 clientid 是由 JS 生成的客户端 id,username 是输入的账号,只是前面多了两个字符 e5,这两个字符不是本章的重点。

该请求发送以后,服务端返回的响应是一个 json。其中 data 字段就是验证码图片经过 base64 编码之后的数据。如果要确定是不是需要的验证码图片,就需要剔除 data:image/jpeg;base64,它后面的数据才是图片,复制



图 11-1 某网站的验证码图片

后用 Base64 解码。input 字段是验证码的备选项,可以看出该选项的位置分布是随机的。

```
{ "code": 0, "model": 0, "data": "data: image\/jpeg; base64, \\/9j\/4AAQSkZJRgABAQAA...", "input":
["H", "U", "D", "I", "Y", "T", "len": 2]}
```

接着查看服务器进行结果验证的请求 /yzmtest/check. php? t=1596249644221, 这是一个 POST 请求,提交的数据如下所示。其中, password 字段是明文, data 字段是选择的验证码备选项处理以后的结果。

```
clientid = gl03ushm0yw&data = 624964140912496413098211125&username = e51111111&password
= 22222222
```

data 字段的值有两种来源: 服务器返回, 该值在抓到的所有数据包中可以找到。另外一种就是由 JS 生成的。通过目前抓到的数据来分析, 并没有找到 data 的值, 因此猜测是由 JS 生成的。

Chrome 开发者工具中抓到的数据包, 会把发送当前请求时的函数堆栈放在 Initiator 这一栏中, 把鼠标放上去就会显示。如图 11-2 所示, @ 的前面是函数名, 后面表示源代码在哪个文件中。其中, jquery 是开源的第三方库, 在算法分析中一般最后才看 jquery 文件。单击 lebo. yzm. pc. min. js 文件, 会转到 Sources 面板, 可以发现这个文件有 JS 混淆。在本章的后续内容中只截取代码的关键部分进行展示。也可以直接通过 Chrome 开发者工具去动态调试混淆后的 JS 文件来分析算法, 只不过比较耗时。本章选择还原这个 JS 文件。

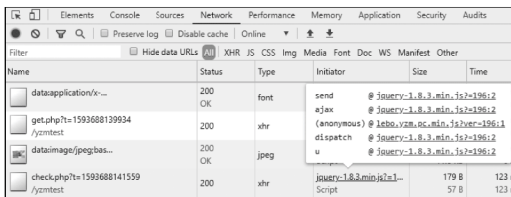


图 11-2 工具抓到的数据包

11.1.2 数组乱序

通过 11.1.1 节的分析, 得知生成 data 字段的关键代码都是混淆过的。既然要还原 JS 混淆后的代码, 就要先分析该网站中使用了哪些混淆方案。下面介绍原始代码中的数组乱序, 这里只截取部分展示。代码如下:

```
var_0x5a19 = ['iBjbgFzCz0iYnV0dG9uIHdoaXRlIj48Y2FudmFzIHdpZHRoPS11MHB4IiBoZWlnaH09IjQwHgiIHNoewxlPSJ3aWR0aDo1MHB4O2hlaWdodDo0MHB4IiBpZD0iYnRuY2Fudl8 = ', ...];
```



```

(function (_0x5cb8ac, _0x23b38e) {
    //该函数用于还原大数组的真实顺序,当前只是定义,并未执行
    var _0x524cab = function (_0x18d3aa) {
        while (--_0x18d3aa) {
            _0x5cb8ac['push'](_0x5cb8ac['shift']());
        }
    };
    var _0x4821c6 = function () {
        var _0x398fd3 = { ...
            //该函数用于内存爆破,具体原理在本书的第6章中已经详细介绍
            'setCookie': function (_0x34eee4, _0x5586ea, _0x5ef311, _0x855d4e) {
                ...
            },
            'removeCookie': function () {
                return 'dev';
            },
            'getCookie': function (_0x296452, _0x39f9cd) { ...
                //在这个函数中调用前面定义的 _0x524cab 函数,来还原大数组的真实顺序
                var _0x3d1943 = function (_0xelf36d, _0x41a83b) {
                    _0xelf36d(++_0x41a83b);
                };
                var _0x3d1943(_0x524cab, _0x23b38e); ...
            }
        };
        //该函数用于检测代码是否格式化,就是检测前面定义的 _0x398fd3 对象中的 removeCookie
        var _0xef60f0 = function () {
            //实例化一个正则对象,把 removeCookie 方法转为字符串后,测试能否匹配
            var _0x4ba6f5 = new RegExp('\\x5cw+\\x20*\\x5c(\\x5c)\\x20*{\\x5cw+ ...}');
            return _0x4ba6f5['test'](_0x398fd3['removeCookie']['toString']());
        };
        _0x398fd3['updateCookie'] = _0xef60f0;
        var _0x4f27d5 = '';
        //执行前面定义好的 _0xef60f0 函数
        var _0x4e6c47 = _0x398fd3['updateCookie']();
        if (!_0x4e6c47) {
            _0x398fd3['setCookie'](['*', 'counter', 0x1];
        } else if (_0x4e6c47) {
            _0x4f27d5 = _0x398fd3['getCookie'](null, 'counter');
            ...
        };
        _0x4821c6();
    }(_0x5a19, 0x144));

```

在代码的最开始处,定义了一个名为 _0x5a19 的全局大数组,这个数组的顺序实际上是打乱的。接着还有一个匿名自执行的函数,该函数的作用为还原数组的真实顺序、检测代码是否格式化以及内存爆破。

匿名自执行函数首先定义了一个用于还原大数组真实顺序的 _0x524cab 函数,该函数在 _0x398fd3 对象的 getCookie 方法中被调用。之后定义了 _0x4821c6 函数,然后调用它。

在 0x4821c6 函数中,先定义了 _0x398fd3 对象,然后定义了 _0xef60f0 函数,接着把 _0xef60f0 函数赋值给了 _0x398fd3 对象的 updateCookie,最后调用 updateCookie 执行代码格式化的检测。如果检测到代码格式化,就进行内存爆破,否则就调用 _0x398fd3 对象的 getCookie 方法来还原数组真实顺序。数组顺序还原后的结果为:

```
var _0x5a19 = [ 'Mnw5fDN8MTB8MTJ8MhwxNXw3FDZ8NhwxFDh8MTZ8NXwxMXwxN3wxM3wxNhwxOA == ',
  'MTd8MTV8NhwxMHw1fDE2fDE0fDB8OHw5fDN8MTB8MTJ8MXwyfDd8MTN8NnwxMQ == ', ... ];
```

11.1.3 字符串加密

示例代码如下:

```
var _0x1f20d3 = {
  'oscqj': _0x2ba9('0x0'),
  'fkUhC': _0x2ba9('0x1'),
  'lyKzh': _0x2ba9('0x2'),
  'PrDEx': function (_0x56ee68, _0x1607ce) {
    return _0x56ee68(_0x1607ce);
  },
  'bIzJA': function (_0x32a55a, _0x4df6cd) {
    return _0x32a55a + _0x4df6cd;
  },
  'QxklZ': _0x2ba9('0x3'),
  'evllC': _0x2ba9('0x4'),
  'XhIuf': _0x2ba9('0x5'),
  'VBKmx': function (_0x1a8117, _0x3e3864) {
    return _0x1a8117 * _0x3e3864;
  },
  ...
}
```

原始代码中有大量的字符串加密。其中 _0x2ba9 函数就是字符串解密函数,该函数接收一个字符串参数,函数定义部分位于 11.1.2 节介绍的匿名自执行函数之后,代码如下:

```
var _0x2ba9 = function (_0x101b8f, _0xcd7c6f) {
  //将传进来的字符串转为数值
  _0x101b8f = _0x101b8f - 0x0;
  //把该数值当作索引,取出大数组中对应的密文字符串赋值给 _0x27941a
  var _0x27941a = _0x5a19[_0x101b8f];
  //在 JS 中函数也是对象,可以有自己的属性和方法
  //因为字符串解密函数会多次调用,给函数定义一个 LmvHXr 属性,避免解密函数多次初始化
  if (_0x2ba9['LmvHXr'] === undefined) { ...
    //真正的解密函数是一个 Base64 解码
    _0x2ba9['dzogWA'] = function (_0x30b4a5) {
      var _0x4f6190 = atob(_0x30b4a5); ...
    };
  }
```

```

        //该对象用来存放解密后的字符串
        _0x2ba9['nKWcry'] = {};
        _0x2ba9['LmvHXr'] = '!![]';
    }
    //根据索引,从 nKWcry 对象中取出已经解密的明文字符串,如果不存在,再去解密
    var _0x578a10 = _0x2ba9['nKWcry'][_0x101b8f];
    if (_0x578a10 === undefined) { ...
        //用来检测是否格式化
        new _0x4b1809(_0x2ba9)['OsLPar']();
        //把密文字符串传入函数中解密
        _0x27941a = _0x2ba9['dzoqWA'](_0x27941a);
        //将解密后的字符串按索引存入对象中,下次调用时根据索引直接取明文字符串
        _0x2ba9['nKWcry'][_0x101b8f] = _0x27941a;
    } else {
        _0x27941a = _0x578a10;
    }
    return _0x27941a;
};

```

可以看出, `_0x2ba9` 解密函数的参数是大数组中的密文字符串索引。把密文字符串进行 Base64 解码后就可以得到明文字符串。然后用明文字符串替换对应的调用 `_0x2ba9` 解密函数的地方。例如, `_0x5a19` 数组下标为 0 的成员 Base64 解密后的字符串为 '2|9|3|10|12|0|15|7|6|4|1|8|16|5|11|17|13|14|18', 那么字符串加密的还原只需要用该字符串替换掉所有 `_0x2ba9('0x0')` 的地方即可。

接下来学习 `_0x2ba9` 解密函数中的格式化检测代码。代码如下:

```

var _0x4b1809 = function (_0x3bd14) {
    this['YlKlnG'] = _0x3bd14;
    this['NsTJKl'] = [0x1, 0x0, 0x0];
    this['HILlKx'] = function () {
        return 'newState';
    };
    this['GGmyeM'] = '\x5cw\x20*\x5c(\x5c)\x20*{\x5cw\x20*';
    this['VUtdVO'] = '[\x27|\x22]. + [\x27|\x22];?\x20*';
};
//检测是否格式化函数的定义部分
_0x4b1809['prototype']['OsLPar'] = function () {
    //实例化一个正则对象,通过匹配 HILlKx 函数来检测格式化
    var _0x1403ab = new RegExp(this['GGmyeM'] + this['VUtdVO']);
    var _0x3fad0 = _0x1403ab['test'](this['HILlKx']()['toString'])();
    -- this['NsTJKl'][0x1] : -- this['NsTJKl'][0x0];
    //调用 anWLTR 函数来判断格式化的检测结果
    return this['anWLTR'](_0x3fad0);
};
//没有格式化,直接返回 _0x26db32,格式化了就调用内存爆破函数 xTDWoN
_0x4b1809['prototype']['anWLTR'] = function (_0x26db32) {
    if (!Boolean(~_0x26db32)) {
        return _0x26db32;
    }
}

```

```

    }
    return this['xTDWoN'](this['YlKlnG']);
};
//内存爆破函数
_0x4b1809['prototype']['xTDWoN'] = function (_0x597ca7) {
    for (var _0x3e27c4 = 0x0, _0x192434 = this['NsTJKl']['length']; _0x3e27c4 < _0x192434;
    _0x3e27c4++) {
        this['NsTJKl']['push'](Math['round'](Math['random']({})));
        _0x192434 = this['NsTJKl']['length'];
    }
    return _0x597ca7(this['NsTJKl'][0x0]);
};
//调用检测函数
new _0x4b1809(_0x2ba9)['OsLPar']();

```

11.1.4 花指令

花指令的形式有很多种,花指令混淆原理在第6章中有详细介绍。这里以一个例子来说明问题,代码如下:

```

_0x22b277[_0x2ba9('0xd4')][_0x22b277[_0x2ba9('0xd5')], new Date()][_0x2ba9('0x96')]({})

// _0x2ba9('0xd4') 解密后为 'etrcq'
// _0x2ba9('0xd5') 解密后为 'oiFIc'
// _0x2ba9('0x96') 解密后为 'getTime'
// 因此上述代码可以简化为
// _0x22b277['etrcq'](_0x22b277['oiFIc'], new Date()['getTime'])()

```

上述代码化简后,有一个字符串已经是明文了,但是另外两个字符串_0x22b277['etrcq']和_0x22b277['oiFIc'],还需要继续寻找。

```

var _0x22b277 = {
    ...
    'etrcq': function (_0x3fb552, _0x5f9394) {
        return _0x1f20d3[_0x2ba9('0x7e')][_0x3fb552, _0x5f9394];
    },
    ...
    'oiFIc': _0x1f20d3[_0x2ba9('0x7f')],
    ...
}
// _0x2ba9('0x7e') 解密后为 'ZFmTI'
// _0x2ba9('0x7f') 解密后为 'a0JeX'
// 因此又要去找到 _0x1f20d3['ZFmTI'] 和 _0x1f20d3['a0JeX']
var _0x1f20d3 = {
    ...
    'ZFmTI': function (_0x4e4bbb, _0x19e1fa) {

```

```

        return _0x4e4bbb + _0x19elfa;
    },
    'a0JeX': _0x2ba9('0x2e'),
    ...
}
// _0x2ba9('0x2e') 解密后为 '/yzmtest/get.php?t = '

```

通过上述的分析可以看出, `_0x22b277['etrcq']` 是一个函数, 它把传入的两个参数相加。`_0x22b277['oiFIc']` 是一个字符串, 所以代码化简过程如下:

```

_0x22b277[_0x2ba9('0xd4')]( _0x22b277[_0x2ba9('0xd5')], new Date()[_0x2ba9('0x96')]())

_0x22b277['etrcq'](_0x22b277['oiFIc'], new Date()[ 'getTime']())

'/yzmtest/get.php?t = ' + new Date()[ 'getTime']()

```

化简后的代码简洁明了, 本书把类似 `_0x22b277['oiFIc']` 的混淆称为字符串花指令, 把类似 `_0x22b277['etrcq']` 的混淆称为函数花指令。从这个例子可以看出, 剔除花指令之前需要先把字符串解密。

11.1.5 流程平坦化

为了更好地说明问题, 先把字符串加密以及花指令处理掉后, 再截取其中部分代码来展示。还原前的代码如下:

```

$( "#LoginForm, #LoginForm2" )[ "unbind" ]( "submit" )[ "bind" ]( "submit", function () {
    var _0x27fd60 = "1|2|4|7|5|3|8|0|6"[ "split" ]( '|' ),
        _0x46c0ac = 0x0;
    while ( !![] ) {
        switch ( _0x27fd60[ _0x46c0ac++ ] ) {
            case '0':
                yzmObj2[ "load" ]();
                continue;
            case '1':
                var _0xe696fb = $( this );
                continue;
            case '2':
                var _0x94eld7 =
                    _0xe696fb[ "find" ]( "input[ name = 'username' ]" )[ "val" ]();
                continue;
            case '3':
                CkType = _0xe696fb[ "attr" ]( 'id' );
                continue;
            case '4':
                var _0x104aba = _0xe696fb[ "find" ]( "input[ name = 'passwd' ]" )[ "val" ]();
                continue;

```

```

case '5':
    if (yzmObj2 != null && $(this)["attr"]('id') == CkType) {
        if (yzmObj2["getstatus"]() == 0x1)
            return ![];
    }
    continue;
case '6':
    return ![];
case '7':
    if (_0x94eld7 == '' || _0x94eld7 == '账号') {
        alert("\u8BF7\u8F93\u5165\u767B\u5165\u5E10\u53F7!!");
        return ![];
    } else if (_0x104aba == '' || _0x104aba == "xx@x@x.x") {
        alert("\u8BF7\u8F93\u5165\u5BC6\u7801!!");
        return ![];
    } else if (_0x104aba["length"] > 0x0 && _0x104aba["length"] < 0x6) {
        alert("\u5BC6\u7801\u957F\u5EA6\u4E0D\u80FD\u5C11!!");
        return ![];
    }
    continue;
case '8':
    yzmObj2 = $['fn']["yzmbox"](_0xe696fb,
        $("#leboyzm")["text"]() + _0x94eld7, _0x104aba);
    continue;
}
break;
}
});

```

其中,第3行代码中的"1|2|4|7|5|3|8|0|6"["split"](',')就是一个分发器,根据这个分发器中的数据,按顺序来执行对应的 case 代码块。即代码混淆后,打乱了原先的代码执行顺序。还原后的代码如下所示:

```

$("#LoginForm, #LoginForm2")["unbind"]("submit")["bind"]("submit", function () {
    var _0xe696fb = $(this);
    var _0x94eld7 = _0xe696fb["find"]("input[name = 'username']")["val"]();
    var _0x104aba = _0xe696fb["find"]("input[name = 'passwd']")["val"]();
    if (_0x94eld7 == '' || _0x94eld7 == '账号') {
        alert("\u8BF7\u8F93\u5165\u767B\u5165\u5E10\u53F7!!");
        return ![];
    } else if (_0x104aba == '' || _0x104aba == "xx@x@x.x") {
        alert("\u8BF7\u8F93\u5165\u5BC6\u7801!!");
        return ![];
    } else if (_0x104aba["length"] > 0x0 && _0x104aba["length"] < 0x6) {
        alert("\u5BC6\u7801\u957F\u5EA6\u4E0D\u80FD\u5C11!!");
        return ![];
    }
}
if (yzmObj2 != null && $(this)["attr"]('id') == CkType) {
    if (yzmObj2["getstatus"]() == 0x1)

```

```
        return ![];
    }
    CkType = _0xe696fb["attr"]('id');
    yzmObj2 = $[ 'fn' ][ "yzmbox" ]( _0xe696fb, $ ( " # leboyzm" ) [ "text" ] ( ) + _0x94e1d7, _0x104aba );
    return ![];
});
```

对于有前端经验的开发者来说,这个逻辑看上去很清晰,在后端替换后能动态调试的情况下,这个逻辑会更清晰。由此可见,还原JS代码对于逆向分析极其重要。



视频讲解

11.2 还原代码中的常量

11.2.1 整体代码结构

代码的整体结构如下所示。首先使用 require 加载必要的依赖,然后读取JS文件中的代码,用 parser 组件解析成 ast,接着对 ast 进行一系列的操作,最后把 ast 转为字符串,保存到新文件。

```
const parser = require("@babel/parser");
const traverse = require("@babel/traverse").default;
const t = require("@babel/types");
const generator = require("@babel/generator").default;
const fs = require('fs');

const jscode = fs.readFileSync('./astDemo.js', {
  encoding: "utf-8"
});
let ast = parser.parse(jscode);

//此处对 ast 进行一系列的操作

let code = generator(ast).code;
fs.writeFile('./demoNew.js', code, (err) => {});
```

11.2.2 字符串解密与去除数组混淆

根据前面几节的分析可知,原始代码在最开始处定义了一个大数组,紧接着定义了一个用于还原数组顺序的匿名自执行的函数,然后定义了一个字符串解密的函数。而要调用字符串解密函数,又必须先得到大数组和用于还原数组顺序的匿名自执行函数。下面是原始代码的整体结构:

```
console.log(ast.program.body);
/*
```

```

Node {type: 'VariableDeclaration', ... declarations: Array(1)}
Node {type: 'ExpressionStatement', ... expression: Node}
Node {type: 'VariableDeclaration', ... declarations: Array(1)}
Node {type: 'ExpressionStatement', ... expression: Node}
Node {type: 'VariableDeclaration', ... declarations: Array(1)}
Node {type: 'VariableDeclaration', ... declarations: Array(1)}
Node {type: 'VariableDeclaration', ... declarations: Array(1)}
Node {type: 'ExpressionStatement', ... expression: Node}
*/

```

要完成字符串解密,先要得到原始代码中的三个部分,即大数组、还原数组顺序的函数和字符串解密函数,其中的字符串解密函数,还需要得到字符串解密函数的名字。

```

//得到解密函数所在节点
let stringDecryptFuncAst = ast.program.body[2];
//得到解密函数的名字
let DecryptFuncName = stringDecryptFuncAst.declarations[0].id.name;
//新建一个 AST,把原始代码中的大数组、还原数组顺序的函数和字符串解密函数,加入到 body 节点中
let newAst = parser.parse('');
newAst.program.body.push(ast.program.body[0]);
newAst.program.body.push(ast.program.body[1]);
newAst.program.body.push(stringDecryptFuncAst);
//把上述三部分代码转为字符串,由于存在格式化检测,需要指定选项来压缩代码
let stringDecryptFunc = generator(newAst, {compact: true}).code;
//将字符串形式的代码执行,这样就可以在 nodejs 中运行解密函数了
eval(stringDecryptFunc);

```

再次强调,由于原始代码中存在格式化检测和内存爆破的代码,所以上述代码在生成字符串代码时,需要指定选项,使用压缩后的代码来执行,否则会内存溢出。

现在 nodejs 中已经有解密函数了,接下去可以直接计算节点,如 `_0x2ba9('0x0')`,并用结果替换它。代码如下:

```

traverse(ast, {
  //遍历所有变量
  VariableDeclarator(path){
    //当变量名与解密函数名相同时,就执行相应操作
    if(path.node.id.name == DecryptFuncName){
      let binding = path.scope.getBinding(DecryptFuncName);
      binding && binding.referencePaths.map(function(v){
        v.parentPath.isCallExpression() &&
        v.parentPath.replaceWith(t.stringLiteral(eval(v.parentPath + '')));
      });
    }
  }
});
//字符串解密以后,原始代码的三个部分就没用了,可以删去
ast.program.body.shift();

```



```

ast.program.body.shift();
ast.program.body.shift();

/* _0x2ba9('0xa1') 在 AST Explorer 网站中解析后的结构为
Node {
  type: 'CallExpression',
  ...
  callee: Node { type: 'Identifier', ... name: '_0x2ba9'},
  arguments: [
    Node { type: 'StringLiteral', ... value: '0xa1'}
  ]
}
*/

```

上述代码中遍历的是 VariableDeclarator 节点,当 name 与 DecryptFuncName 一致时,就执行相应的解密和替换操作。先获取 DecryptFuncName 的 binding,然后通过 binding 的 referencePaths 属性,就可以获取到所有引用处的 Path 对象(里面存放的都是 Identifier 类型),接着通过数组的 map 方法来遍历它们。因为在原始代码中,给 _0x2ba9 解密函数增加了几个属性(在 11.1.3 节中有提到),所以这些属性也会出现在 referencePaths 中,但是字符串解密不在 referencePaths 中进行,所以需要判断遍历节点的父节点。如果父节点是 CallExpression,则需要对该节点进行处理。处理方法很简单,把节点转为字符串代码,然后通过 eval 计算结果,把结果用 stringLiteral 包装,替换掉父节点,也就是 CallExpression 节点。由于之前已经把解密函数的定义取出,因此当前 nodejs 中是有该解密函数的定义的,不会报错。

以原始代码中的某段代码为例,还原前后的代码分别为:

```

//还原前
this[_0x2ba9('0x9a')] = _0x22b277[_0x2ba9('0xb4')](Math[_0x2ba9('0xa4')]( _0x22b277[_0x2ba9('0x93')](Math[_0x2ba9('0x94')]()), 0x5)), 0x5);
//还原后
this["$strlen"] = _0x22b277["hdpPm"] (Math["floor"](_0x22b277["cdTWA"])(Math["random"]()), 0x5)), 0x5);

```



视频讲解

11.3 剔除花指令

11.3.1 花指令剔除思路

示例代码如下:

```

var _0x1f20d3 = { ...
  'ZFmTI': function (_0x4e4bbb, _0x19elfa) {
    return _0x4e4bbb + _0x19elfa;
  },

```

```
'aOJeX': '/yzmtest/get.php?t = ', ...
}
var _0x22b277 = { ...
  'etrqc': function (_0x3fb552, _0x5f9394) {
    return _0x1f20d3[ 'ZFmTI' ](_0x3fb552, _0x5f9394);
  },
  'oiFIc': _0x1f20d3[ 'aOJeX' ], ...
}
// _0x22b277[ 'oiFIc' ] 字符串花指令
// _0x22b277[ 'etrqc' ] 函数花指令
```

花指令剔除可以分为以下两种情况。

1. 字符串花指令的剔除

对于字符串花指令 `_0x22b277[ 'oiFIc' ]`，可以遍历所有 `MemberExpression` 节点，取出 `object` 节点名和 `property` 节点值。在 `ObjectExpression` 节点中找到对应的值，如果类型为 `MemberExpression`，就说明还需要继续找，再次取出 `object` 节点名和 `property` 节点值，并在 `ObjectExpression` 节点中找到对应的值，以此类推，直到找到的值类型为 `StringLiteral`，进行替换即可，这个过程需要用到递归。

2. 函数花指令的去除

对于函数花指令 `_0x22b277[ 'etrqc' ]`，遍历所有 `MemberExpression` 节点，取出 `object` 节点名和 `property` 节点值。在 `ObjectExpression` 节点中找到对应的值，如果类型为 `FunctionExpression` 并且函数体内部有 `MemberExpression` 节点，就说明还需要继续找，直到找到类型为 `FunctionExpression` 并且函数体内部没有 `MemberExpression` 节点的节点，才是最终需要的节点。

在 `ObjectExpression` 节点中找到对应的值有一个比较简便的方式，可以在 `nodejs` 中定义一个 `totalObj` 对象，然后解析原始代码中所有的 `ObjectExpression`，加入到 `totalObj` 对象中，最后把 `totalObj` 对象变成如下结构：

```
{
  _0x1f20d3: {
    'ZFmTI': Node {...},
    'aOJeX': Node {...},
    'EDRDI': Node {...},
    ...
  },
  _0x22b277: {
    'etrqc': Node {...},
    'oiFIc': Node {...},
    ...
  },
  ...
}
```

在 `nodejs` 中组合出这样的结构后，如果要获取 `_0x22b277[ 'oiFIc' ]` 的定义部分的节

点,只需要使用 `totalObj['_0x22b277']['oiFIc']` 来获取 Node 节点。生成 totalObj 对象的代码如下:

```
var totalObj = {};
function generatorObj(ast){
    traverse(ast, {
        VariableDeclarator(path){
            //init 节点为 ObjectExpression 时,它就是需要处理的对象
            if(t.isObjectExpression(path.node.init)){
                //取出对象名
                let objName = path.node.id.name;
                //以对象名作为属性名在 totalObj 中创建对象
                objName && (totalObj[objName] = totalObj[objName] || {});
                //解析对象中的每一个属性,加入到新建的对象中去,注意属性值依然是 Node 类型
                totalObj[objName] && path.node.init.properties.map(
                    function(p){
                        totalObj[objName][p.key.value] = p.value;
                    });
            }
        }
    });
    return ast;
}
ast = generatorObj(ast);

Node {
  type: 'VariableDeclarator', ...
  id: Node { type: 'Identifier', ... name: '_0x1f20d3' },
  init: Node { type: 'ObjectExpression', ...
    properties: [[Node], [Node], [Node], ... ]}
}
//properties 中的每一个 ObjectProperty 结构为
Node {
  type: 'ObjectProperty', ...
  method: false,
  key: Node { type: 'StringLiteral', ... value: 'NmhQU' },
  computed: false,
  shorthand: false,
  value: { type: 'StringLiteral', value: 'submit', ... }
}
```

11.3.2 字符串花指令的剔除

字符串花指令比较容易剔除,遍历 ObjectExpression 节点的 properties 属性,每得到一个 ObjectProperty 的 value 值,都递归找到真实的字符串后进行替换。代码如下:

```

    traverse(ast, {
      VariableDeclarator(path){
        if(t.isObjectExpression(path.node.init)){
          path.node.init.properties.map(function(p){
            let realNode = findRealValue(p.value);
            realNode && (p.value = realNode);
          });
        };
      }
    });

    /*
      var _0x1f20d3 = { 'a0JeX': '/yzmtest/get.php?t=', ... }
      var _0x22b277 = { 'oiFIc': _0x1f20d3['a0JeX'], ... }
      //处理后变为如下形式,不管代码中调用哪个对象中的属性,都是真实字符串,最后统一替换
      var _0x1f20d3 = { 'a0JeX': '/yzmtest/get.php?t=', ... }
      var _0x22b277 = { 'oiFIc': '/yzmtest/get.php?t=', ... }
    */

```

接下来实现 `findRealValue`, 根据前面对原始代码的分析可知, `ObjectProperty` 的 `value` 值有三种类型: `MemberExpression`、`FunctionExpression` 和 `StringLiteral`。先不处理 `FunctionExpression` 节点, `StringLiteral` 节点是真实的字符串, 所以也不用处理, 只需要处理 `MemberExpression` 节点。实现的代码如下:

```

function findRealValue(node){
  if(t.isMemberExpression(node)){
    let objName = node.object.name;
    let propName = node.property.value;
    if(totalObj[objName][propName]){
      return findRealValue(totalObj[objName][propName]);
    }else{
      return false;
    }
  }else{
    return node;
  }
}

//剔除字符串花指令以后,更新 totalObj 对象
ast = generatorObj(ast);

/*
  var _0x1f20d3 = { 'a0JeX': '/yzmtest/get.php?t=', ... }
  var _0x22b277 = { 'oiFIc': _0x1f20d3['a0JeX'], ... }
*/

```

假如传给 `findRealValue` 的参数为 `_0x1f20d3 [ 'a0JeX' ]` 节点, 它是一个 `MemberExpression` 节点, 因此会取出 `'_0x1f20d3'` 赋值给 `objName`, 取出 `'a0JeX'` 赋值给 `propName`, 然后获取 `totalObj['_0x1f20d3']['a0JeX']` 中存放的 `Node` 节点, 继续传入到

findRealValue 中进行递归。这时候传入的是 '/yzmtest/get.php? t='，并非成员表达式，因此直接返回原节点。一直返回，最后执行 `realNode && (p.value = realNode)` 进行节点替换。

上述过程只处理了 ObjectExpression 节点的属性值，最后还要对代码中引用的地方都进行替换（替换之前更新 totalObj 对象）。实现的代码如下：

```

    traverse(ast, {
      MemberExpression(path){
        let objName = path.node.object.name;
        let propName = path.node.property.value;
        totalObj[objName] && t.isStringLiteral(totalObj[objName][propName]) && path.replaceWith(
          (totalObj[objName][propName]);
        }
      });

```

遍历所有 MemberExpression 节点，取出 objName 和 propName，只要对应节点在 totalObj 对象中存在，并且类型为 StringLiteral，就进行替换。

以原始代码中经过字符串解密以后的某段代码为例，还原前后的代码分别为：

```

//还原前
var _0x167f85 = _0x1f20d3["oscqj"]["split"]('|'), _0x57c351 = 0x0;
//还原后
var _0x167f85 = "2|9|3|10|12|0|15|7|6|4|1|8|16|5|11|17|13|14|18["split"]('|'), _0x57c351 = 0x0;

```



视频讲解

11.3.3 函数花指令的剔除

函数花指令的剔除原理与字符串花指令的剔除相似，只不过需要更改递归的函数。实现的代码如下：

```

    traverse(ast, {
      VariableDeclarator(path){
        if(t.isObjectExpression(path.node.init)){
          path.node.init.properties.map(function(p){
            let realNode = findRealFunc(p.value);
            realNode && (p.value = realNode);
          });
        }
      });
    function findRealFunc(node){
      if(t.isFunctionExpression(node) && node.body.body.length == 1){
        let expr = node.body.body[0].argument.callee;
        if(t.isMemberExpression(expr)){
          let objName = expr.object.name;

```

```
        let propName = expr.property.value;
        if(totalObj[objName]){
            return findRealFunc(totalObj[objName][propName]);
        }else{
            return false;
        }
    }
    return node;
}
}
}
//剔除函数花指令以后,更新 totalObj 对象
ast = generatorObj(ast);

/*
var _0x1f20d3 = {
    'ZFmTI': function (_0x4e4bbb, _0x19elfa) {
        return _0x4e4bbb + _0x19elfa;
    },
    'EDRDI': function(_0x247161, _0x4c41eb) {
        return _0x247161(_0x4c41eb);
    }, ...
};
var _0x22b277 = {
    'etrqc': function (_0x3fb552, _0x5f9394) {
        return _0x1f20d3['ZFmTI'](_0x3fb552, _0x5f9394);
    }, ...
};
*/
```

以上述代码中的注释为例,将它们的特点归纳为以下 4 点。

- (1) 它们都是 FunctionExpression 节点。
- (2) 函数体只有一个 ReturnStatement 节点。
- (3) ReturnStatement 节点的 argument 属性类型如果不是 CallExpression,那么该节点就是最终需要的节点,例如上述注释中的 ZFmTI 函数。
- (4) ReturnStatement 节点的 argument 属性类型如果是 CallExpression,但是其 callee 属性类型不是 MemberExpression,那么该节点也是最终需要的节点,例如上述注释中的 EDRDI 函数。

综上所述,findRealFunc 函数最终需要递归的节点的特点为: ReturnStatement 节点的 argument 属性类型为 CallExpression,并且 CallExpression 节点的 callee 属性类型为 MemberExpression。

findRealFunc 的实现方式为:

- (1) 筛选出类型为 FunctionExpression,且函数体只有一行代码的节点,其余节点直接返回原节点。
- (2) 取出 ReturnStatement 节点的 argument 属性中的 callee。

(3) 如果第 2 步得到的节点类型为 MemberExpression, 就取出对象名和属性名, 从 totalObj 中获取对应的节点, 传入 findRealFunc 函数进行递归。

(4) 如果第 2 步得到的是 undefined, 就说明不是 CallExpression 节点, 该节点已经是最终需要的节点, 直接返回原节点即可。

(5) 如果第 2 步得到的不是 undefined, 也不是 MemberExpression 节点, 该节点已经是最终需要的节点, 直接返回原节点即可。

(6) 一直返回, 执行 realNode &&. (p. value = realNode) 替换节点。

上述过程只处理 ObjectExpression 节点的属性值, 处理后的效果如下:

```
var _0x1f20d3 = {
  'ZFmTl': function (_0x4e4bbb, _0x19elfa) {
    return _0x4e4bbb + _0x19elfa;
  },
  'EDRDI': function (_0x247161, _0x4c41eb) {
    return _0x247161(_0x4c41eb);
  },
  ...
};
var _0x22b277 = {
  'etrqc': function (_0x3fb552, _0x5f9394) {
    return _0x1f20d3['ZFmTl'](_0x3fb552, _0x5f9394);
  },
  ...
};
/* 上面的代码被修改为
var _0x1f20d3 = {
  'ZFmTl': function (_0x4e4bbb, _0x19elfa) {
    return _0x4e4bbb + _0x19elfa;
  },
  'EDRDI': function (_0x247161, _0x4c41eb) {
    return _0x247161(_0x4c41eb);
  },
  ...
};
var _0x22b277 = {
  'etrqc': function (_0x4e4bbb, _0x19elfa) {
    return _0x4e4bbb + _0x19elfa;
  },
  ...
};
*/
```

假如代码中的引用为 _0x22b277['etrqc'](1000, 2000), 这时应当遍历 CallExpression 节点, 判断 callee 的节点类型为 MemberExpression, 去 totalObj 对象中查找对应代码节点。取出该代码节点里面的 ReturnStatement 中的 argument 属性, 然后判断节点类型, 如果是 BinaryExpression, 就取出其中的 operator 属性, 与传入的实参 1000 和 2000 构建新的 BinaryExpression 节点, 并替换整个 _0x22b277['etrqc'](1000, 2000)。

假如代码中的引用为 `_0x1f20d3['EDRDI'](_0x247162, 2000)`, 这时应当遍历 `CallExpression` 节点, 判断 `callee` 的节点类型, 如果为 `MemberExpression`, 就去 `totalObj` 对象中查找对应代码节点。取出该代码节点里面的 `ReturnStatement` 中的 `argument` 属性, 然后判断节点类型, 如果为 `CallExpression`, 就用传入的实参构建一个新的 `CallExpression` 节点, `_0x247162` 作为 `callee`, `2000` 作为 `argument`, 来替换掉原先的整个节点。

因此, 最后对代码中引用的地方进行替换(替换之前记得更新 `totalObj` 对象)的代码如下:

```

    traverse(ast, {
      CallExpression(path) {
        // callee 不为 MemberExpression 的节点, 不做处理
        if (!t.isMemberExpression(path.node.callee)) return;
        // 取出对象名和属性名
        let objName = path.node.callee.object.name;
        let propertyName = path.node.callee.property.value;
        // 如果在 totalObj 中有相应节点, 就需要进行替换
        if (totalObj[objName] && totalObj[objName][propertyName]) {
          // totalObj 中存放的是函数节点
          let myFunc = totalObj[objName][propertyName];
          // 在原始代码中, 函数体只有 return 语句, 取出其中的 argument 节点
          let returnExpr = myFunc.body.body[0].argument;
          // 判断 argument 节点类型, 并且用相应的实参来构建二项式或者调用表达式
          // 然后替换当前遍历到的整个 CallExpression 节点
          if (t.isBinaryExpression(returnExpr)) {
            let binExpr = t.binaryExpression(returnExpr.operator,
              path.node.arguments[0], path.node.arguments[1]);
            path.replaceWith(binExpr);
          } else if (t.isCallExpression(returnExpr)) {
            // 把 arguments 数组中的下标为 1 和以后的成员放入 newArray 中
            let newArray = path.node.arguments.slice(1);
            let callExpr = t.callExpression(path.node.arguments[0], newArray);
            path.replaceWith(callExpr);
          }
        }
      }
    });
    // 花指令删除后, 就可以删除原始代码中的 ObjectExpression. 最好先判断是否有引用, 如果没有引用, 就删除
    traverse(ast, {
      VariableDeclarator(path) {
        if (t.isObjectExpression(path.node.init)) {
          path.remove();
        }
      }
    });
  });

```

以原始代码中经过字符串解密以后的某段代码为例, 还原前后的代码分别为:


```
//还原前
this["$ $ strlen"] = _0x22b277["hdpPm"](Math["floor"](_0x22b277["cdTwA"])(Math["random"]()),
0x5)), 0x5);
//还原后
this["$ $ strlen"] = Math["floor"](Math["random"]()) * 0x5) + 0x5;
```



视频讲解

11.4 还原流程平坦化

11.4.1 获取分发器

还原流程平坦化之前,应当先进行字符串解密以及剔除花指令。在还原流程平坦化的过程中,需要先获取分发器,因为分发器中记录着代码原先的真实顺序。以 11.1.5 节中的代码为例,分发器在 AST 中的结构如下:

```
Node {
  type: 'MemberExpression',
  ...
  object: Node { type: 'StringLiteral', ... value: '1|2|4|7|5|3|8|0|6' },
  property: Node { type: 'StringLiteral', ... value: 'split' },
  computed: true
}
```

可以看出,遍历 MemberExpression 节点,如果其中的 object 节点为 StringLiteral 类型,property 节点为 StringLiteral 类型,并且 value 为 'split',就是分发器所在的 MemberExpression 节点。其中 path、node、object、value 是记录代码原先真实顺序的字符串。因此,获取分发器的代码如下:

```
traverse(ast, {
  MemberExpression(path) {
    if (t.isStringLiteral(path.node.object) &&
      t.isStringLiteral(path.node.property, { value: 'split' })) {
      console.log(path.node.object.value);
    }
  }
});
// '1|2|4|7|5|3|8|0|6'
```

11.4.2 解析 switch 结构

把 AST 中的 switch 结构解析到 nodejs 的数组中。这样在复原代码顺序时,就可以快速、准确地获取对应的代码节点。想要解析整个 switch,先要学习 WhileStatement 节点和 SwitchStatement 节点的结构,以 11.1.5 节中的代码为例:

```
//WhileStatement 的结构
Node {
  type: 'WhileStatement', ...
  test: Node {
    type: 'UnaryExpression', ...
    operator: '!',
    prefix: true,
    argument: Node {
      type: 'UnaryExpression', ...
      operator: '!',
      prefix: true,
      argument: [Node]
    }
  },
  body: Node { ... }
}
```

WhileStatement 节点的 test 属性是 while 循环的条件, body 属性是 while 循环体中的代码, body 体中的结构如下:

```
Node {
  type: 'BlockStatement', ...
  body: [Node {
    type: 'SwitchStatement', ...
    discriminant: Node { type: 'MemberExpression', ...
      object: Node { type: 'Identifier', ... name: '_0x27fd60' },
      property: Node {
        type: 'UpdateExpression', ...
        operator: '++',
        prefix: false,
        argument: [Node]
      },
    },
    computed: true
  },
  cases: [
    Node {
      type: 'SwitchCase', ...
      consequent: [Array],
      test: Node { type: 'StringLiteral', ... value: '0' }
    },
    ...
  ],
  Node { type: 'BreakStatement', ...label: null }
],
  directives: []
}
```

SwitchStatement 节点中的 discriminant 属性是 switch 语句中用来控制跳转的表达式。

cases 属性中存放着 switch 中所有的 case 代码块。SwitchCase 节点的 test 属性是 case 后的值,consequent 属性是具体的代码块。

解析 switch 结构的代码,如下所示:

```

traverse(ast, {
  MemberExpression(path) {
    if (t.isStringLiteral(path.node.object) &&
        t.isStringLiteral(path.node.property, { value: 'split' })) {
      //找到类型为 VariableDeclaration 的父节点
      let varPath = path.findParent(function (p) {
        return t.isVariableDeclaration(p);
      });
      //获取下一个同级节点
      let whilePath = varPath.getSibling(varPath.key + 1);
      //解析 switch 结构
      let myArr = [];
      whilePath.node.body.body[0].cases.map(function (p) {
        myArr[p.test.value] = p.consequent[0];
      });
    }
  }
});

```

上述代码首先是沿用 11.4.1 节中的代码,定位到分发器所在的 MemberExpression 节点。接着要定位到 switch 节点,才能解析整个 switch 节点。这里的方案是先找到 WhileStatement 节点,原始代码在 AST 中的整体结构,如下所示:

```

Node {
  type: 'VariableDeclaration',
  ...
  declarations: [
    Node {
      type: 'VariableDeclarator',
      ...
      id: Node { type: 'Identifier', ... name: '_0x27fd60'},
      init: Node { type: 'CallExpression', ...
        callee: Node {
          type: 'MemberExpression',
          ...
          object: Node { type: 'StringLiteral', ... value: '1|2|4|7|5|3|8|0|6'},
          property: Node { type: 'StringLiteral', ... value: 'split' },
          computed: true
        },
        arguments: [ [Node] ]
      }
    },
    Node { type: 'VariableDeclarator' ... }
  ],
}

```

```
    kind: 'var'
  }
  Node { type: 'WhileStatement', ... }
```

可以看出, WhileStatement 节点是 VariableDeclaration 节点的下一个同级节点, 而当前定位到的是分发器所在的 MemberExpression 节点。因此在本节最开始的代码中, 通过 path.findParent 找到分发器所在节点的类型为 VariableDeclaration 的父节点, 赋值给 varPath。然后通过 varPath.getSibling 获取到下一个同级节点。

接着通过 WhileStatement 节点, 找到其下属的 switch 节点中的 cases 数组, 也就是存放着所有 case 代码块的节点。最后遍历 cases 数组, 用 case 后面的值作为数组索引, case 中具体的代码节点作为对应的数组的成员, 把整个 switch 解析到预先定义的 myArr 数组中。

11.4.3 复原语句顺序

接下来复原代码顺序, 实现的代码如下所示:

```
let parentPath = whilePath.parent;
varPath.remove();
whilePath.remove();
// path.node.object.value 取到的是 '1|2|4|7|5|3|8|0|6'
let shufferArr = path.node.object.value.split("|");
shufferArr.map(function (v) {
    parentPath.body.push(myArr[v]);
});
```

首先找到 WhileStatement 的父节点, 也就是 BlockStatement 节点。然后把分发器所在的节点和 WhileStatement 节点移除, 相当于把 BlockStatement 的 body 节点清空。然后把存有代码真实顺序的字符串, 分割成数组 shufferArr。遍历该数组, 从之前解析好的 myArr 中, 取出对应索引的代码节点, 添加到 BlockStatement 的 body 节点中。

单个 switch 流程平坦化还原已经完成, 现在要应用到整个 JS 文件中去。由于原始代码中, 存在嵌套的 switch 流程平坦化, 为了防止顺序错乱, 在这里采用每遍历一轮 ast, 只处理一个 switch 流程平坦化的方案, 完整的还原代码如下:

```
//多循环几次, 而不用去判断原始代码中到底有多少个 switch 流程平坦化
for(let i = 0; i < 20; i++){
    traverse(ast, {
        MemberExpression(path){
            if(t.isStringLiteral(path.node.object) &&
                t.isStringLiteral(path.node.property, {value: 'split'})){
                let varPath = path.findParent(function(p){
                    return t.isVariableDeclaration(p);
                });
```

```

        let whilePath = varPath.getSibling(varPath.key + 1);
        let myArr = [];
        whilePath.node.body.body[0].cases.map(function(p){
            myArr[p.test.value] = p.consequent[0];
        });
        let parentPath = whilePath.parent;
        varPath.remove();
        whilePath.remove();
        let shufferArr = path.node.object.value.split("|");
        shufferArr.map(function(v){
            parentPath.body.push(myArr[v]);
        });
        //每遍历一轮 ast,只处理一个 switch 流程平坦化就停止遍历
        path.stop();
    }
}
});
}
}

```

switch 流程平坦化,还原前后的效果,可以参考 11.1.5 节中的代码。其中还有一些十六进制字符串没有还原,还原的方法在第 10 章中有详细介绍,这里不再赘述。

switch 流程平坦化混淆并不是只有这一种。应当掌握原理,在实际应用中根据具体情况进行分析。

11.4.4 协议逆向

还原以后的 JS 文件可以进行静态分析,也可以替换网站中的原文件后,进行动态调试。替换文件可以使用 fiddler 自动响应或者 Chrome Local Overrides 功能,这在第 2 章中有详细介绍。

接下来,就可以对之前抓包过程中找出的几个加密参数,进行逆向分析。在 Chrome 开发者工具的 Sources 面板中,找到替换后的 JS 文件,按 Ctrl+F 键,在弹出的搜索框中输入 "data",找到以下关键代码:

```

var _0x2465df = [],
    _0x44edf3 = new Date()["getTime"]()["toString"]();
_0x974ca9["each"](_0x5bf942, function (_0x4184e0, _0x429229) {
    var _0x1405d0 = _0x974ca9(this)["attr"]('id');
    _0x2465df["push"](_0x1405d0["replace"]("object_", ''));
});
_0x54639d["showloading2"]();
_0x974ca9["ajax"]({
    'url': "/yzmtest/check.php?t=" + _0x44edf3,
    'type': "post",
    'data': {
        'clidntid': _0x54639d[" $ clidntid"],

```

```
'data': _0x2465df["join"]('') + '' + _0x54639d[" $ strlen"] + '' +  
    _0x44edf3["substr"](-0x2) + "1125",  
    'username': _0x54639d[" $ username"],  
    'password': _0x54639d[" $ password"]  
  },  
  ...  
});
```

代码被混淆以后,如果没有做还原,一般搜索不到关键字段。如果想直接进行调试分析,可以使用第2章中介绍的各种调试方法来快速定位,也可以使用第10章介绍的JS Hook来定位关键代码。

从上述代码可知,clientid来自_0x54639d[" \$ clientid"],继续搜索 \$ clientid 赋值的地方,找到如下关键代码:

```
this[" $ clientid"] = Number(Math["random"]()[0]["toString"]()[0]["substr"](-0x3, 0x4) + Date["  
now"]()[0]["toString"](-0x24);
```

clientid是用JS生成的一个随机值,通过多次抓包观察,发现该值在刷新网页时才重新生成。

data由四个部分拼接而成,最后一部分是固定的字符串1125。而_0x44edf3["substr"](-0x2)中的_0x44edf3是new Date()["getTime"]()[0]["toString"](),所以_0x44edf3["substr"](-0x2)的含义是取现行时间戳的后两位。接下来搜索 \$ strlen 赋值的地方,找到如下关键代码:

```
this[" $ strlen"] = Math["floor"](Math["random"]() * 0x5) + 0x5;
```

\$ strlen也是随机生成的。该行代码与之前的this[" \$ clientid"]处于同一个函数中,因此可以猜测该值也是在刷新网页时才重新生成。更换验证码发现该值并不改变,刷新网页发现该值改变,因此猜想是正确的。

接下来分析data参数中最关键的部分:_0x2465df["join"]()。join是数组的方法,数组中加入成员通常使用push方法,因此可以尝试搜索_0x2465df["push"]来找到加入成员的地方。关键代码如下:

```
_0x974ca9["each"](_0x5bf942, function(_0x4184e0, _0x429229) {  
    var _0x1405d0 = _0x974ca9(this)["attr"]('id');  
    _0x2465df["push"](_0x1405d0["replace"]("object_", ''));  
});
```

该JS文件中还使用了jQuery,这里的_0x974ca9["each"]是jQuery提供的方法\$.each。each方法用来遍历数组,它的第一个参数接收一个数组,第二个参数接收一个匿名函数。在该数组遍历过程中,会将数组成员的索引传给匿名函数的第一个参数,会将数组成员传给匿名函数的第二个参数。从上述代码中可以看出,使用each方法遍历_0x5bf942数组,加入到_0x2465df数组的成员是将_0x1405d0中的object_替换为空后的值。_0x1405d0来

自 `_0x974ca9(this)` 中的 `id` 属性。在动态调试状态下,可以直接选中这部分代码,Chrome 开发者工具会计算这个值,如图 11-3 所示。注意,断点必须设置在这部分代码的作用域范围内。

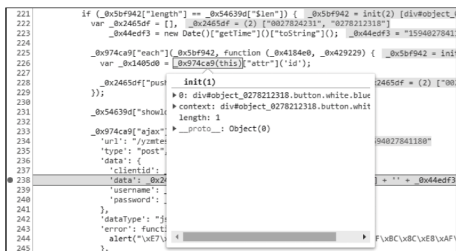


图 11-3 查看表达式信息

从图 11-3 中可以看出, `_0x5b942` 数组中有两个成员,因此会循环两次,该数组中存放的是两个 `div` 元素。`_0x974ca9(this)` 会获取 `_0x5b942` 数组中对应的成员。由于断点设置在 `data` 提交的地方,因此这个循环已经结束, `_0x974ca9(this)` 计算得到的实际上是 `_0x5b942` 数组中下标为 1 的成员。单击显示出来的成员,因为是一个 `HTML` 元素,所以会跳转到 `Elements` 面板,如图 11-4 所示。

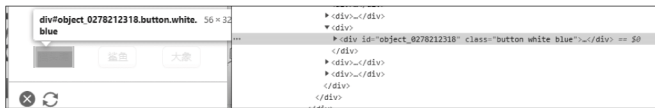


图 11-4 Elements 面板

从图 11-4 中可以看出,它是一个 `div` 元素,并且 `id` 为 `object_0278212318`。选中该元素后,会发现它对应验证码中的其中一个选项。即当单击验证码选项后,会获取对应元素的 `id`,存放到数组中。最后提交时,把数组中的每个成员中的 `object_` 都替换成空,然后与各种值拼接就得到 `data` 的值。`id` 值的来源有两种情况:服务器返回;JS 生成。该案例中是由 JS 生成的,在 JS 文件中搜索 `object_`,找到关键代码如下:

```
_0x974ca9["each"](_0x5283c1, function (_0x4b8168, _0x27f645) {
    _0x54639d[" $ hmdata"]["object_" + _0x27f645['id']] = 0x1;
    _0x4cc8d1["push"]("<div><div id=\"object_" + _0x27f645['id'] + "\" class=\"button
white\">< canvas width=\"50px\" height = \"40px\" style = \"width:50px;height:40px\" id = \"
btn canv_\" + _0x4b8168 + "\"></canvas></div></div>");
});
```

从上述代码中可以看出, `id` 的值来自 `_0x27f645['id']`。这里是用 `each` 方法遍历数组,因此 `_0x27f645['id']` 实际上是从 `_0x5283c1` 数组中获取。在动态调试状态下,鼠标悬停在

_0x5283c1 上,会显示该变量的值,如图 11-5 所示。

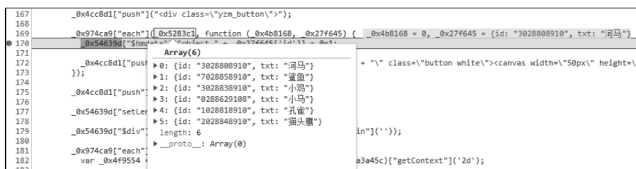


图 11-5 查看变量信息

从图 11-5 中可以看出, _0x5283c1 数组中每一个成员都是对象。对象中有两个属性: id 和 txt。txt 属性是服务器返回的验证码备选项(由于多次调试和重新获取验证码,因此这里的备选项与本章最开始的抓包并不一致)。属性 id 的值是怎么得到的呢? 尝试搜索 _0x5283c1["push"], 找到关键代码如下:

```
_0x974ca9["ajax"]({
  'url': '/yzmtest/get.php?t=' + new Date()["getTime"](),
  'type': 'post',
  'data': {
    'clientid': _0x54639d[" $ clientid"],
    'username': _0x54639d[" $ username"]
  },
  ...
  'success': function (_0x4d7419) {
    ...
    _0x974ca9["each"](_0x4d7419["input"], function (_0x4c0d9a, _0xff7923) {
      _0x5283c1["push"]({
        'id': _0x54639d["getId"](_0x4c0d9a["toString"]()),
        'txt': _0xff7923
      });
    });
  });
});
```

上述代码使用 each 方法遍历 _0x4d7419["input"] 数组,同时从本章最开始的抓包分析中可知, _0x4d7419["input"] 是服务器返回的备选项。_0xff7923 是数组中的成员, _0x4c0d9a["toString"]() 是把数组成员对应的索引转为字符串。也就是说, _0x5283c1 数组中的成员顺序与服务器返回的备选项 input 数组中的顺序是一致的。接着把数组索引传入 _0x54639d["getId"] 函数中,进行下一步处理。搜索 getId, 找到该函数定义处,代码如下:

```
_0x41fffd["prototype"]["getId"] = function (_0x52f0c7) {
  var _0x4f672e = (Math["round"](Math["random"]()) * 0x270f)["toString"]() + new Date()["getTime"]()["toString"]()["substr"](0x4, 0xa) + Math["round"](Math["random"]()) * 0x270f)["toString"]()["toString"]()["substr"](0x3, 0xa);
  return _0x4f672e["substr"](0x0, this[" $ strlen"] - 0x1) + _0x52f0c7 + _0x4f672e["substr"](this[" $ strlen"], _0x4f672e["length"]);
};
```


从上述代码可知,函数体代码总共六行,前四行代码产生一个随机的字符串。后两行代码是对该字符串进行一系列的截取和拼接。但是传入的参数 `_0x52f0c7` 是固定的。它的值是固定的,位置却不固定。在这串随机字符串中,具体哪个是真实的数组索引由 `$strlen` 的值来决定,因此最后提交的 `data` 参数中也拼接了 `$strlen`。其实 `data` 参数并没有加密,只是把选项对应的索引放到一串随机字符串中,以达到混淆的目的。

11.5 小结

还原JS代码之前,需要先弄明白原始代码中使用了哪些混淆手段。当各种混淆手段混合使用时,还原的顺序就显得尤为重要,如果还原的顺序不正确,就可能导致还原失败。一般情况下,应最先处理字符串解密,再还原标识符混淆。

11.6 习题

1. 用递归的方式实现 $5!$ (5 的阶乘)。
2. 编写一段代码,用于检测代码是否格式化。
3. 将 9.5.1 节 `switch` 流程平坦化处理后的代码还原。

图书资源支持

感谢您一直以来对清华版图书的支持和爱护。为了配合本书的使用,本书提供配套的资源,有需求的读者请扫描下方二维码,在图书专区下载,也可以拨打电话或发送电子邮件咨询。

如果您在使用本书的过程中遇到了什么问题,或者有相关图书出版计划,也请您发邮件告诉我们,以便我们更好地为您服务。

我们的联系方式:

地 址: 北京市海淀区双清路学研大厦 A 座 714

邮 编: 100084

电 话: 010-83470236 010-83470237

客服邮箱: 2301891038@qq.com

QQ: 2301891038 (请写明您的单位和姓名)

资源下载: 关注公众号“书圈”下载配套资源。

资源下载, 样书申请



书 圈



获取最新书目



观看课程直播

我和卓斌认识多年，深知其对技术的专注。这是一本少有的爬虫进阶书籍，翻阅本书，相信读者会在爬虫领域有所收获。

—— **r0ysue** 独立安卓安全研究员

和岳阳认识两年来，一直非常佩服他的爬虫能力，更让人赞赏的是，他能将复杂的逻辑、知识通过各种方法拆解，让所有人都能轻松理解。技术能力和教学能力的精进都需要天赋与努力结合，岳阳文理综合的背景完美地将这两个方面融合了起来。对于希望在反爬虫方向精进的读者，这本书是一个非常好的切入点。

—— **毕滢** 清华大学学生创客空间协会创始人

本书由浅入深，叙述了JavaScript逆向的发展和对抗，从开发者、逆向等多维度去思考并且详细介绍了AST混淆等技巧，值得学习。

—— **珍惜** 逆向工作从业者

卓斌的课程由浅入深，既能各维度地补充基础知识，也能深入底层原理。他擅长用通俗易懂的方式来讲解各种复杂性专题，深得学生喜爱，讲课能力一绝。

—— **许炜** 资深爬虫工程师

刮刮卡

资源下载·扩展阅读



书圈

清华大学出版社



官方微信号

ISBN 978-7-302-58517-6



9 787302 585176 >

定价：59.00元